WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case :

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall productivity of the company but also allows for a more agile response to market demands and customer needs.

Requirements :

1. **Salesforce CRM Implementation**
    - Store and manage vehicle details, stock availability, and dealer information in Salesforce.
    - Track customer orders, test drives, and service requests efficiently.
    - Automate workflows to assign orders to the nearest dealer based on customer location.

2. **Process Automation**
    - Prevent order placement if the vehicle is out of stock.
    - Auto-assign orders to the nearest dealer based on the customer's location.
    - Send automated email reminders for scheduled test drives.

3. **Apex and Trigger**

    - Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
    - Use trigger handlers to follow best practices and ensure modularity and maintainability.

4. **Batch Jobs**

    - Develop a batch Apex job to periodically check vehicle stock levels and update availability.
    - Send scheduled email notifications for stock replenishment and order processing.

What you'll learn :

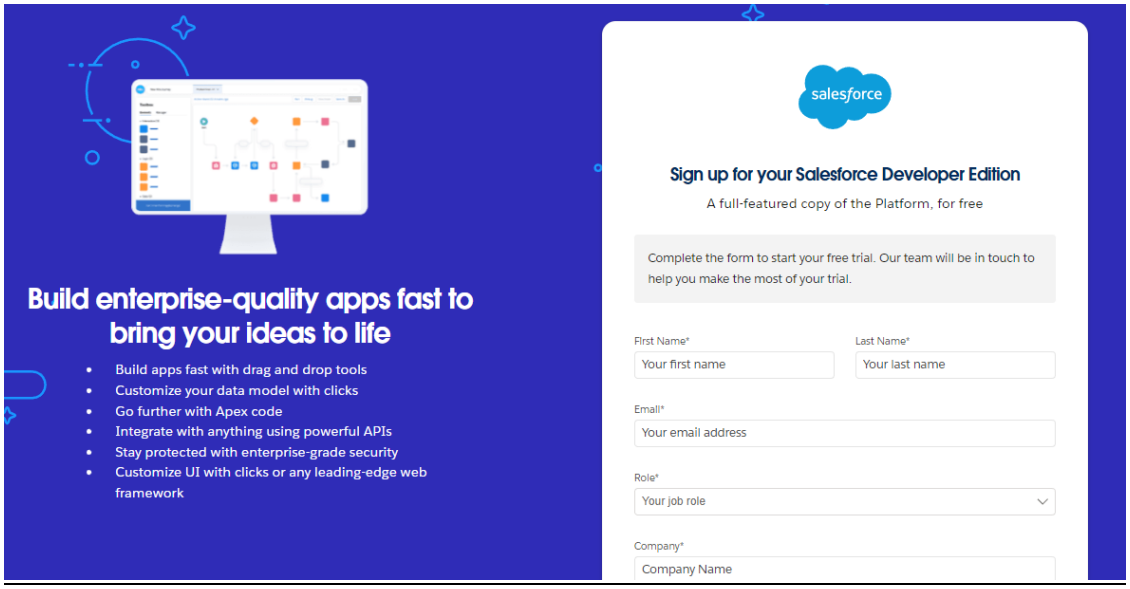
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

MileStone 1 :

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to https://developer.salesforce.com/signup
2. On the sign up form, enter the following details :



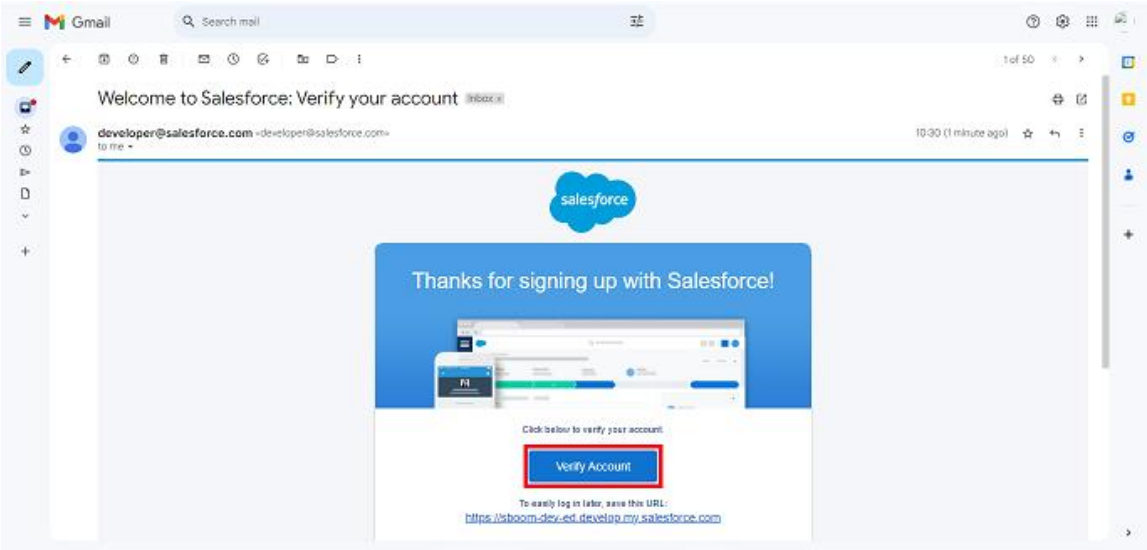
1) First name & Last name
2) Email
3) Role : Developer
4) Company : College Name
5) County : India
6) Postal Code : pin code
7) Username : should be a combination of your name and company

This need not be an actual email id, you can give anything in the format : username@organization.com

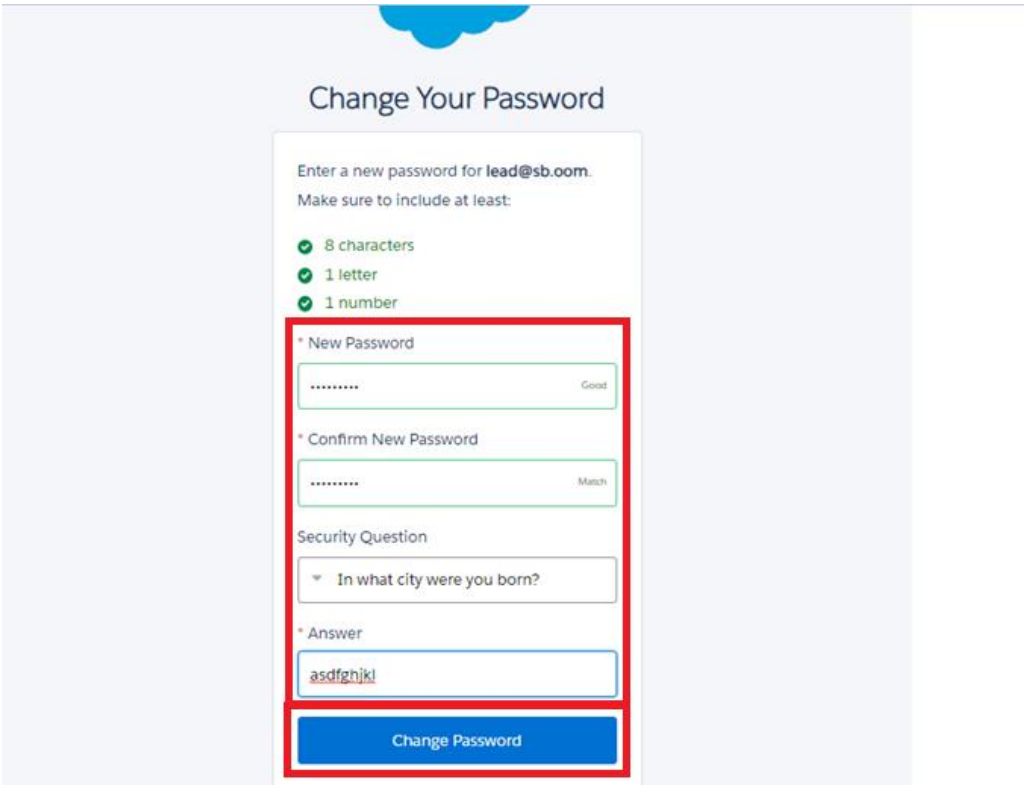
Click on sign me up after filling these.

Activity 2: Account Activation

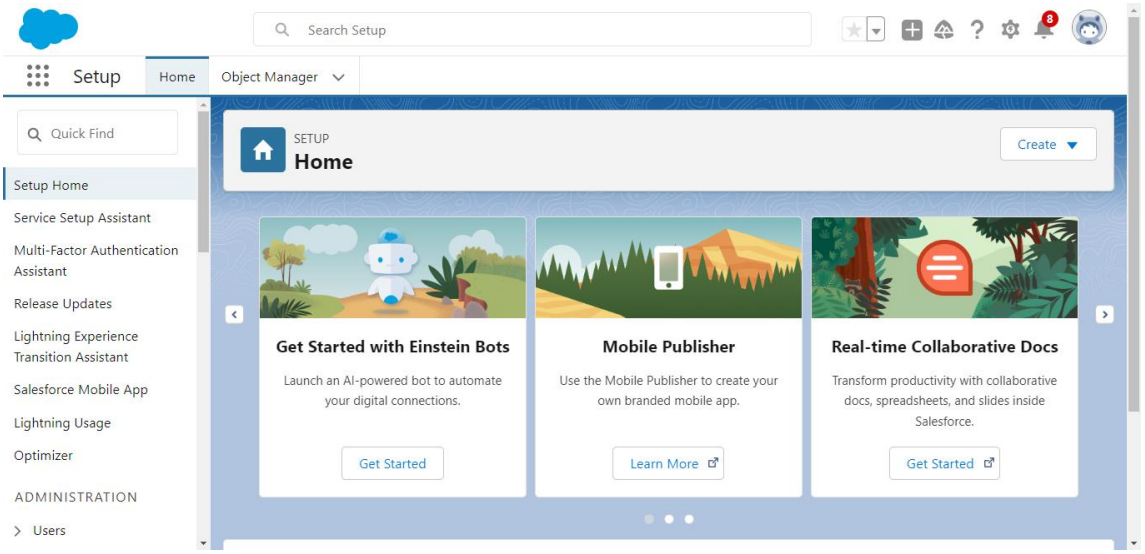
1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.



4. Then you will redirect to your salesforce setup page.

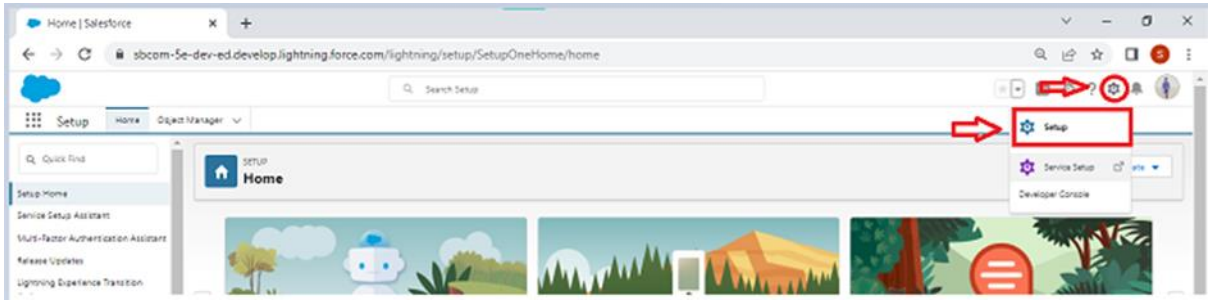


MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:
Click on gear icon → click setup.



Activity 1: Create Vehicle Object

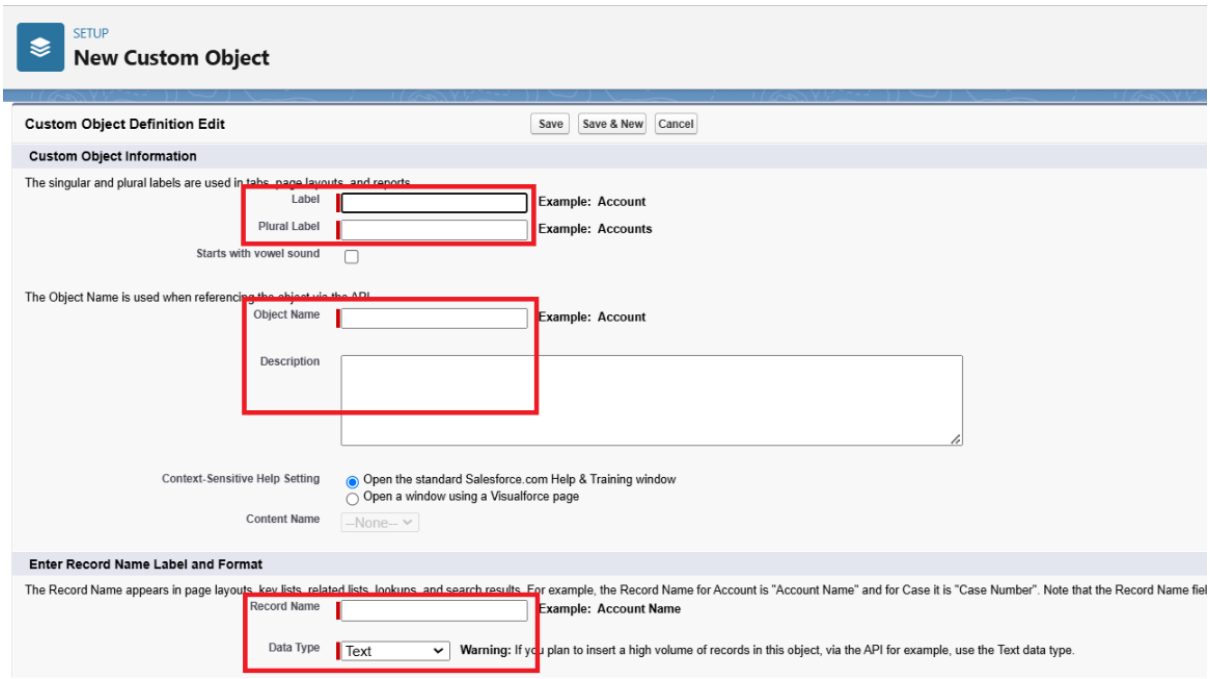
The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

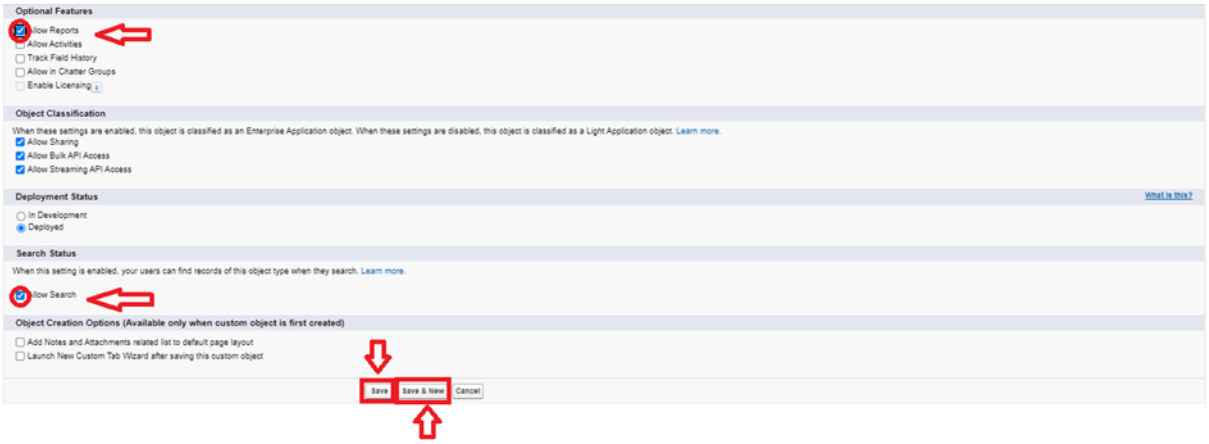
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



1) Enter the label name→ Vehicle
2) Plural label name→ Vehicles
3) Enter Record Name Label and Format
    - Record Name → Vehicle Name
    - Data Type → Text



2. Click on Allow reports,
3. Allow search → **Save.**



Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    1) Enter the label name→ Vehicle Dealer
    2) Plural label name→ Vehicle Dealers
    3) Enter Record Name Label and Format
        - Record Name → Dealer Name
        - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating Customer objects

The purpose of creating a Vehicle Customer object is to have detailed information about Vehicle Customer.

To create an object:

4. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    4) Enter the label name→ Vehicle Customer
    5) Plural label name→ Vehicle Customers
    6) Enter Record Name Label and Format
        - Record Name → Vehicle Customer Name
        - Data Type → Text
5. Click on Allow reports,
6. Allow search → **Save.**

Activity 4: Creating Vehicle Order objects

The purpose of creating a Vehicle Order object is to have detailed information about Vehicle Order.

To create an object:

7. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    7) Enter the label name→ Vehicle Order
    8) Plural label name→ Vehicle Orders
    9) Enter Record Name Label and Format
        - Record Name → Vehicle Order Name
        - Data Type → Text
8. Click on Allow reports,

Allow search → **Save.**

Activity 5: Creating Vehicle Test Drive objects

The purpose of creating a Vehicle Test Drive object is to have detailed information about Vehicle Test Drive.

To create an object:

9. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
10) Enter the label name→ Vehicle Test Drive
11) Plural label name→ Vehicle Test Drives
12) Enter Record Name Label and Format
    - Record Name → Vehicle Test Drive Name
    - Data Type → Text
10. Click on Allow reports,
11. Allow search → **Save.**

Activity 6: Creating Vehicle Service Request objects

The purpose of creating a Vehicle Service Request object is to have detailed information about Vehicle Service Request.

To create an object:

12. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
13) Enter the label name→ Vehicle Service Request
14) Plural label name→ Vehicle Service Requests
15) Enter Record Name Label and Format
    - Record Name → Vehicle Service Request Name
    - Data Type → Text
13. Click on Allow reports,

Allow search → **Save.**

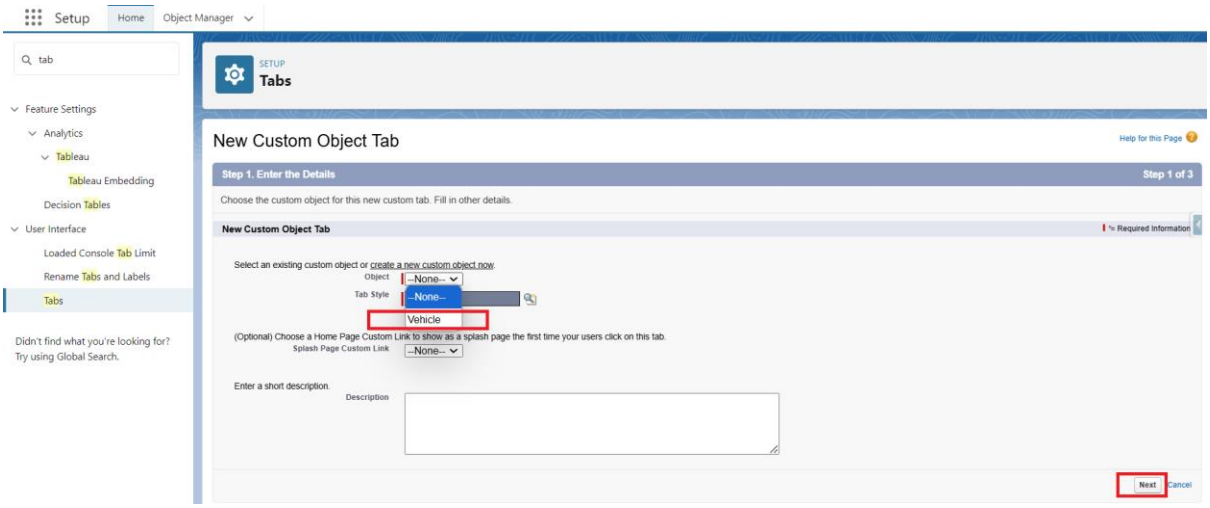
Milestone 3 - Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)

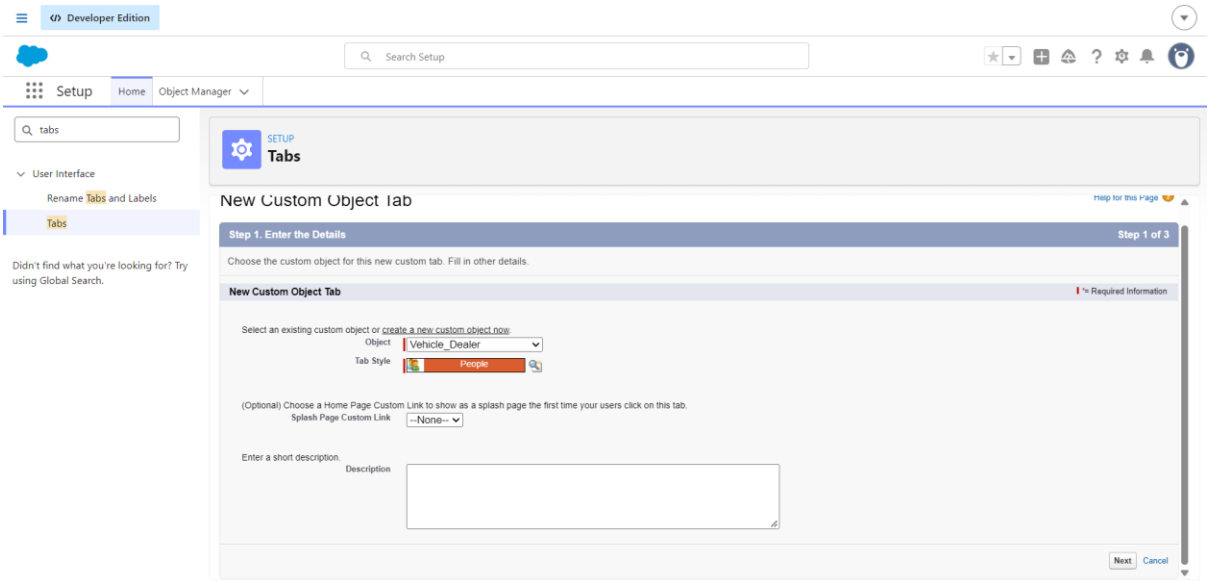
2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default →

Save&New.



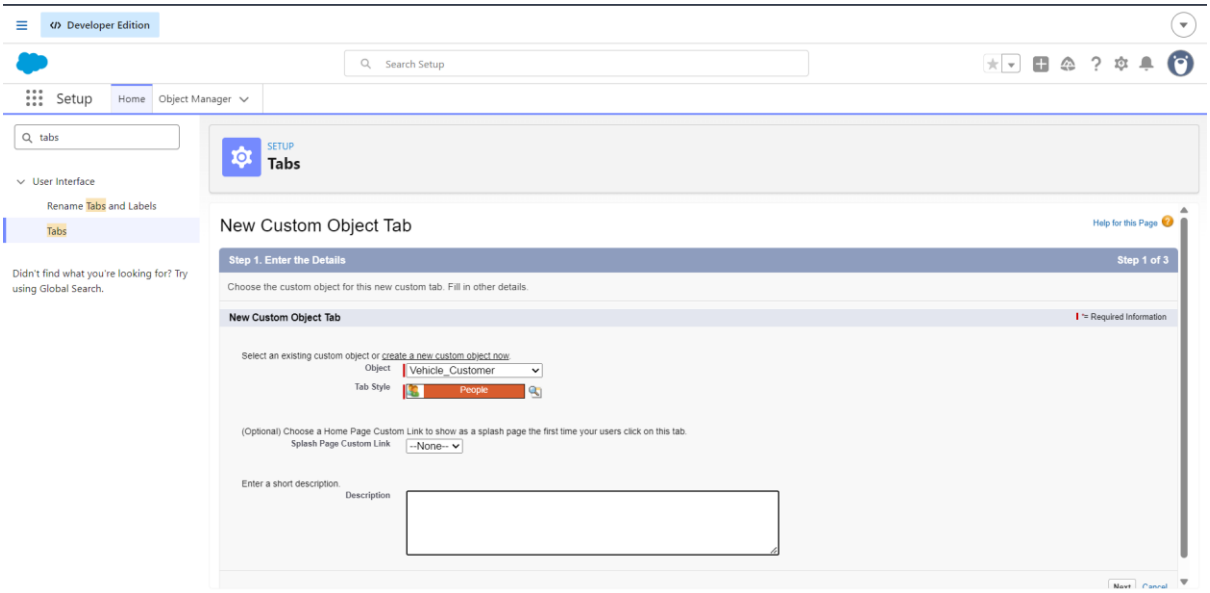
Activity 2: Creating a Custom Tab(Vehicle_Dealer)

3. Select Object(Vehicle_Dealer) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



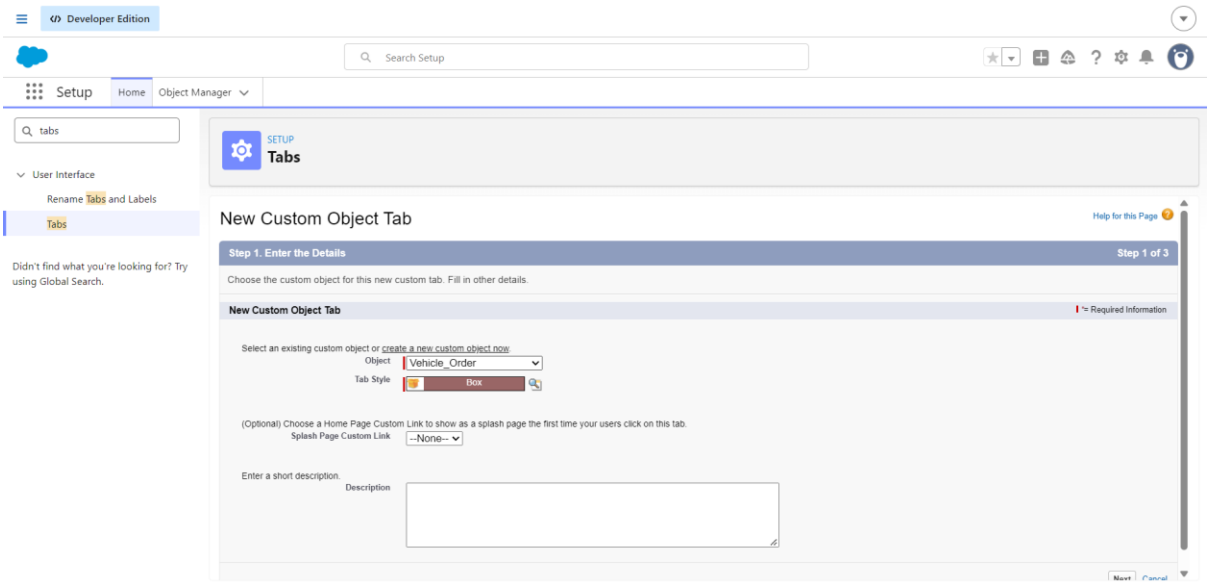
Activity 3: Creating a Custom Tab(Vehicle_Customer)

4. Select Object(Vehicle_Customer) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



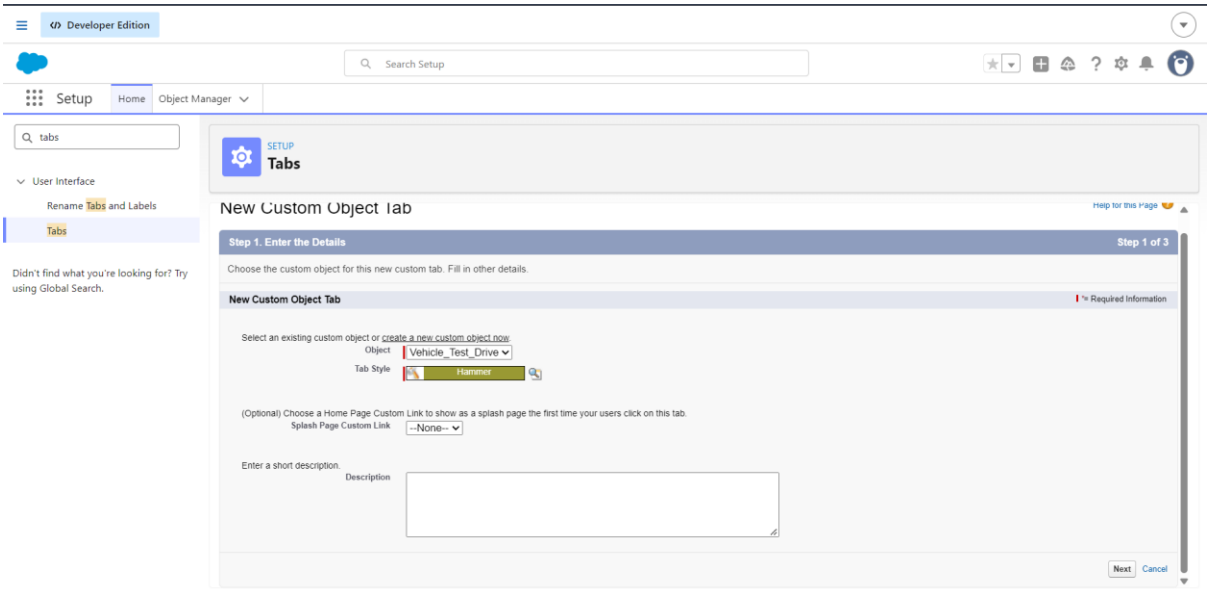
Activity 4: Creating a Custom Tab(Vehicle_Order)

5. Select Object(Vehicle_Order) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



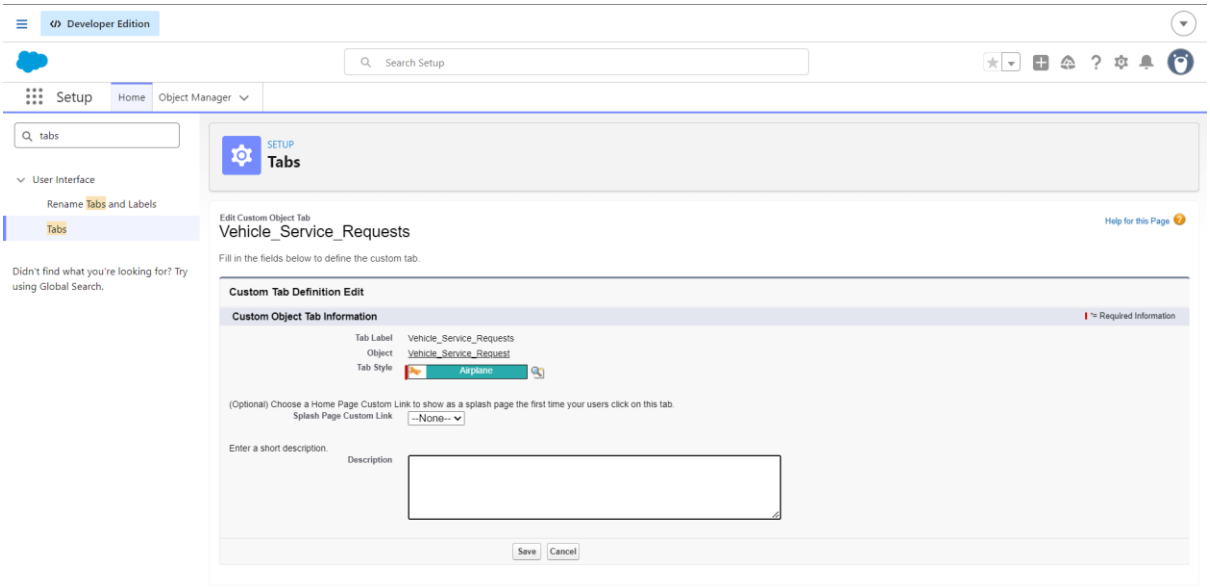
Activity 5: Creating a Custom Tab(Vehicle_Test_Drive)

6. Select Object(Vehicle_Test_Drive) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



Activity 6: Creating a Custom Tab(Vehicle_Service_Request)

7. Select Object(Vehicle_Service_Request) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.

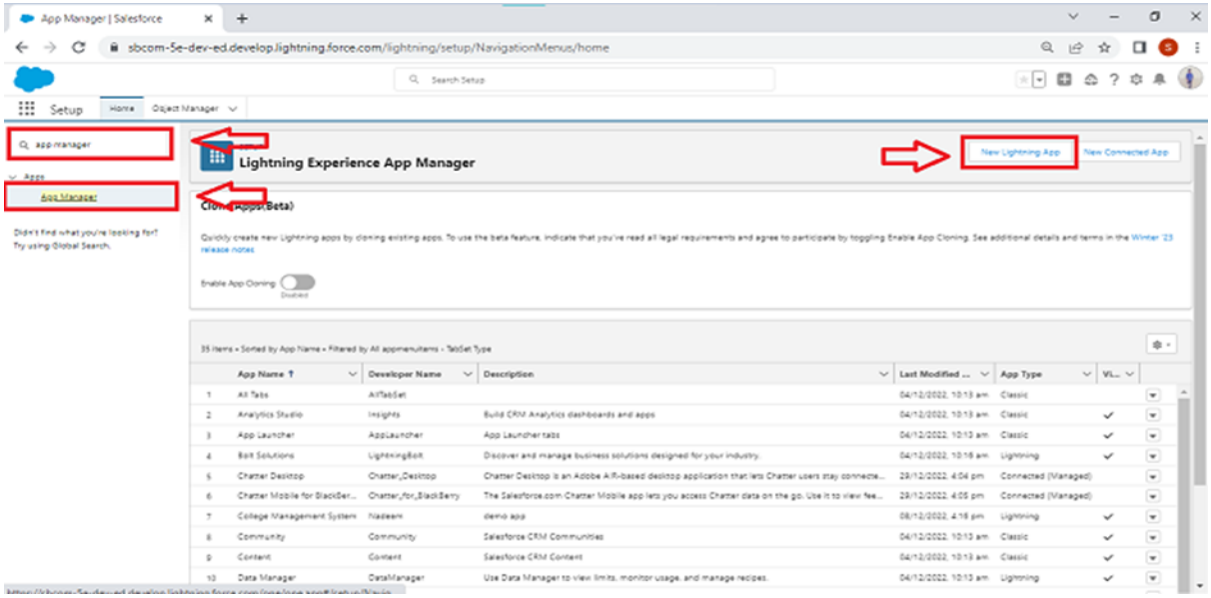


Milestone 4- The Lightning App:

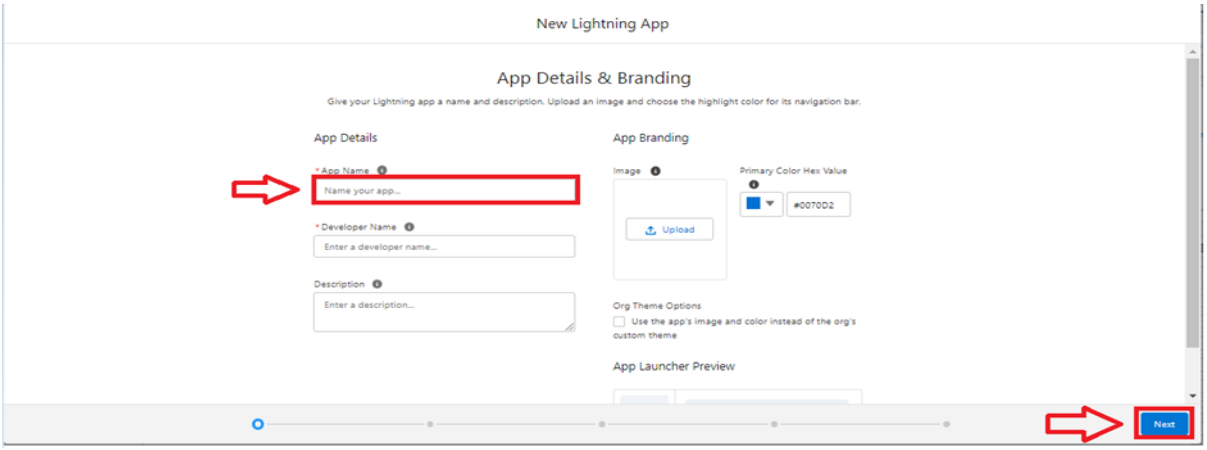
Activity 1: Create a Lightning App

To create a lightning app page:

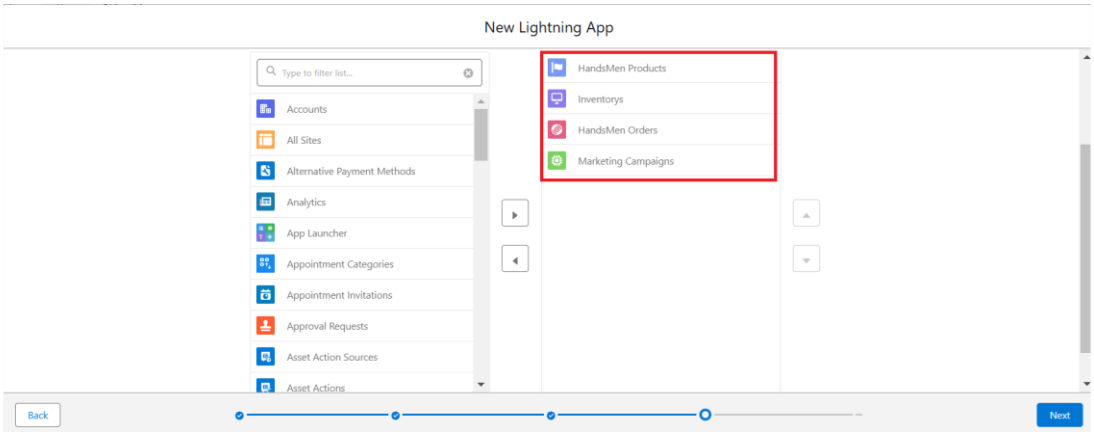
1.Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



1. Fill the app name in app details and branding as follow
   App Name : WhatNext Vision Motors
   Developer Name : this will auto populated
   Description : Give a meaningful description
   Image : optional (if you want to give any image you can otherwise not mandatory)
   Primary color hex value : keep this default
2. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.



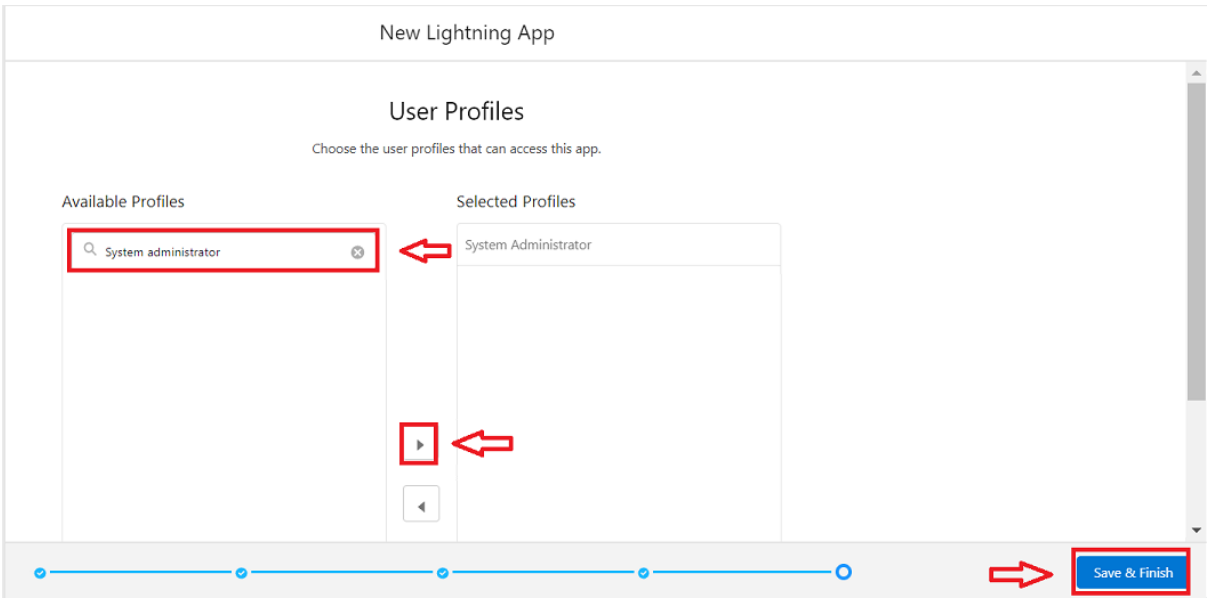
3. To Add Navigation Items:



Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

4. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name**(Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2.Vehicle_ Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

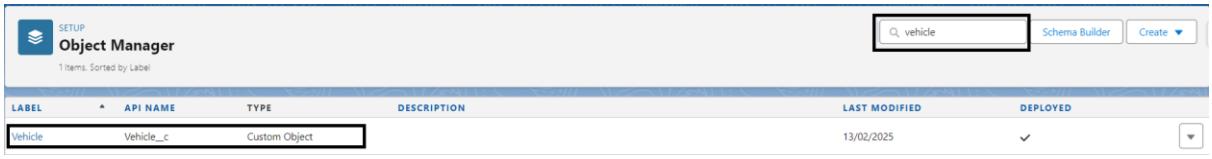
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

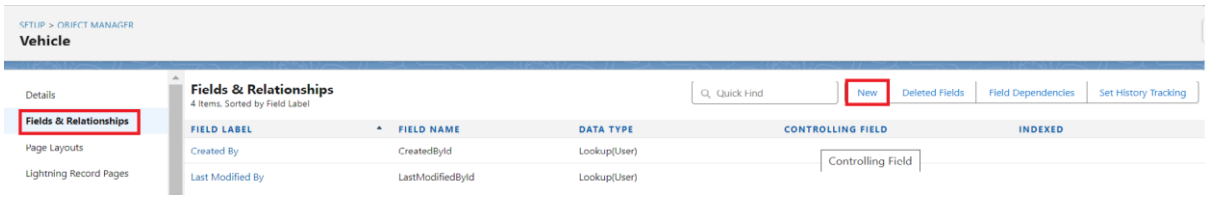
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Activity 1: Creating a Field in Vehicle Object

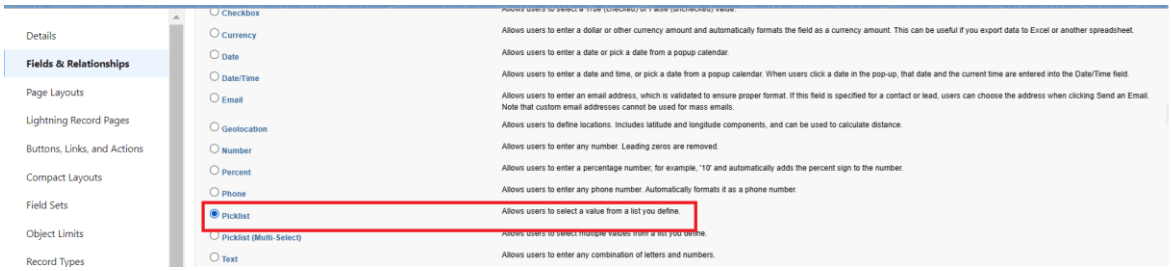
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



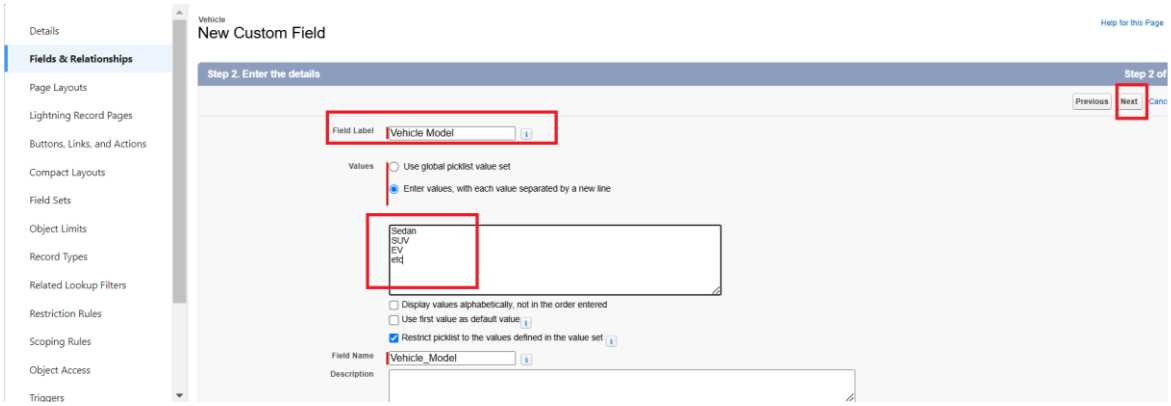
2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



4. Click on Next

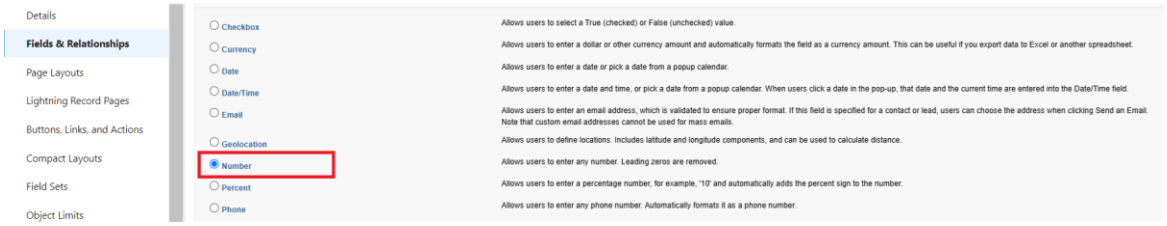


5. Fill the above as following:
   - Field Label: Vehicle Modal
   - Fill PickList Values

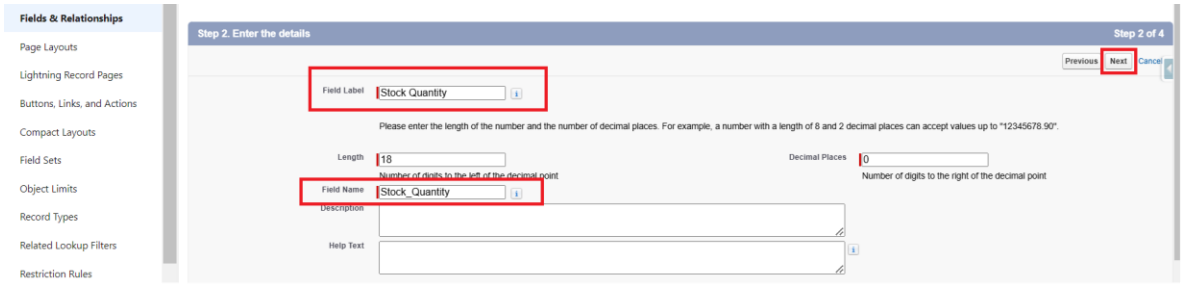
- Click on Next → Next → Save and new.

Activity 2 : Creating Stock Quantity field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.



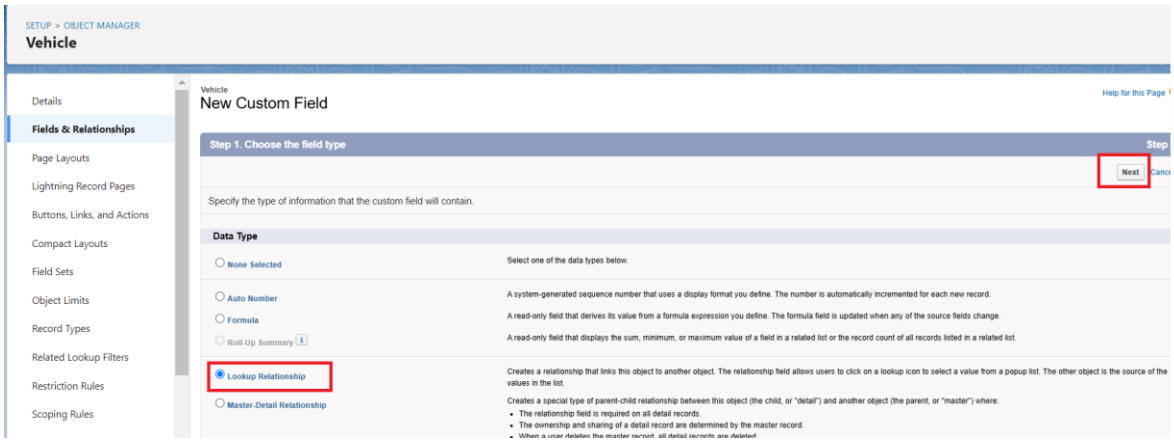
3. Click on Next



4. Fill the above as following:
   - Field Label: Stock Quality
   - Click on Next → Next → Save and new.

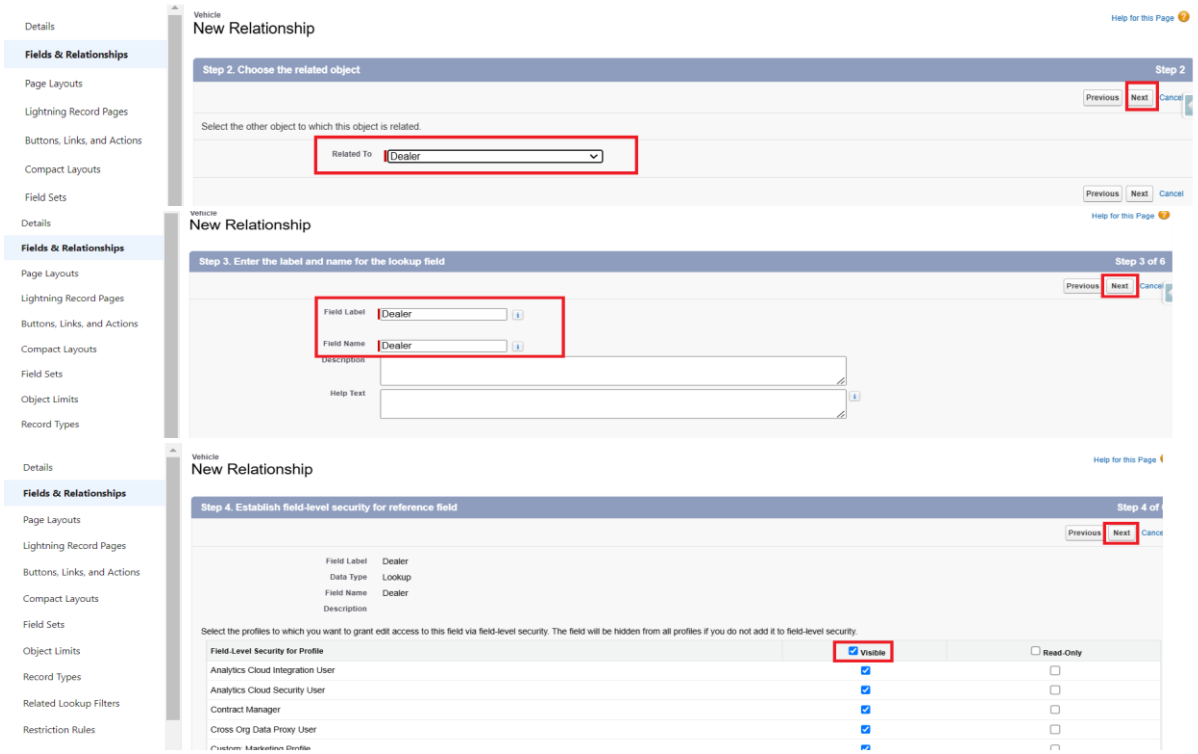
Activity 3 : Creating a Lookup RelationShip in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.



2. Click Next

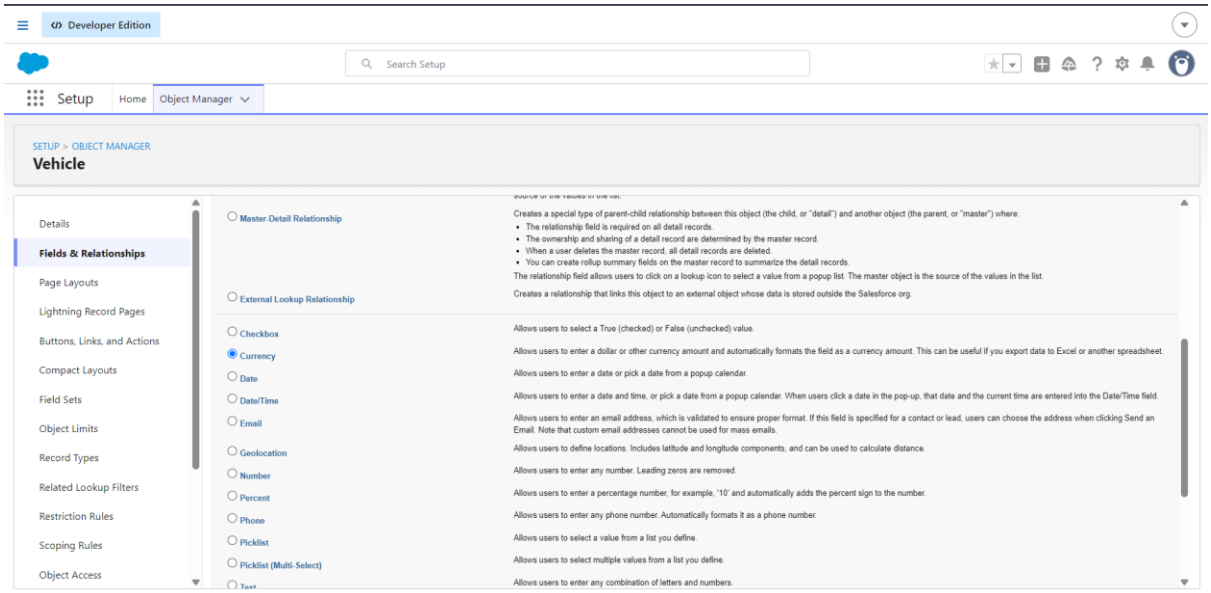
3. Select Dealer Object Form Related Filed.



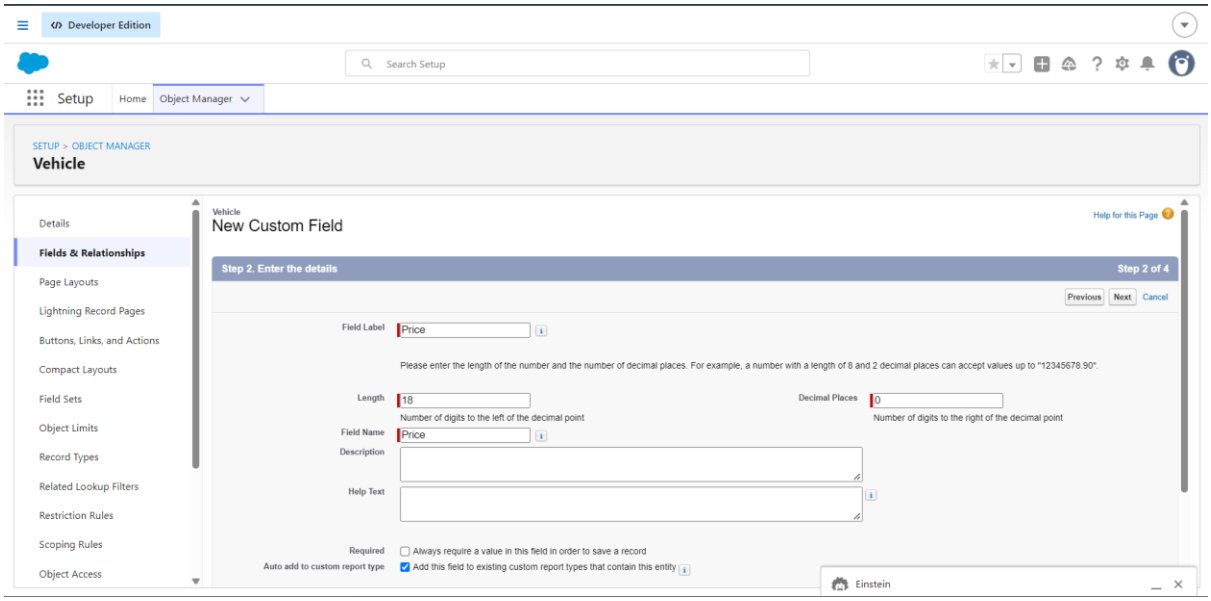
4. Click Next and Save.

Activity 4 : Creating Price field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Currency”.



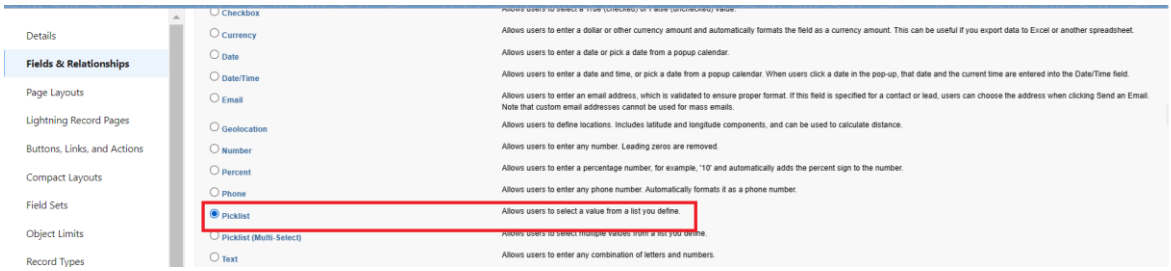
3. Click on Next



4. Fill the above as following:
   - Field Label: Price
   - Click on Next → Next → Save and new.

Activity 5 : Creating Status field in vehicle object

5. Select Data type as “PickList”.



6. Click on Next
7. Fill the above as following:
   - Field Label: Status
   - Fill PickList Values: Available, Out of Stock, Discontinued
8. Click on Next → Next → Save.

Activity 6 : Creating Dealer Name field in vehicle_Dealer object

9. Repeat the steps 1 & 2 from the activity 1
10. Select Data type as “Text”.
11. Click on Next
12. Fill the above as following:
    - Field Label: Dealer_Name
13. Click on Next → Next → Save and new.

Activity 7 : Creating Dealer Location field in vehicle_Dealer object

14. Select Data type as “Text”.
15. Click on Next
16. Fill the above as following:
    - Field Label: Dealer_location
17. Click on Next → Next → Save and new.

Activity 8 : Creating Dealer Code field in vehicle_Dealer object

18. Select Data type as “Auto Number”.
19. Click on Next
20. Fill the above as following:
    - Field Label: Dealer_Code
21. Click on Next → Next → Save and new.

Activity 9 : Creating Phone field in vehicle_Dealer object

22. Select Data type as “Phone”.
23. Click on Next
24. Fill the above as following:
    - Field Label: Phone
25. Click on Next → Next → Save and new.

Activity 10 : Creating Email field in vehicle_Dealer object

26. Select Data type as “Email”.
27. Click on Next
28. Fill the above as following:
    - Field Label: Email
29. Click on Next → Next → Save.

Activity 11 : Creating Vehicle Customer field in vehicle_Order object

30. Repeat the steps 1 & 2 from the activity 1
31. Select Data type as “Lookup Relationship”.
32. Click on Next
33. Fill the above as following:
    - Field Label: Vehicle_Customer
34. Click on Next → Next → Save & New.

Activity 12 : Creating Vehicle field in vehicle_Order object

35. Select Data type as “Lookup Relationship”.

36. Click on Next
37. Fill the above as following:
    - Field Label: Vehicle
38. Click on Next → Next → Save & New.

Activity 13 : Creating Order Date field in vehicle_Order object

39. Select Data type as “Date”.
40. Click on Next
41. Fill the above as following:
    - Field Label: Order_date
42. Click on Next → Next → Save & New.

Activity 14 : Creating Status field in vehicle_Order object

43. Select Data type as “PickList”.
44. Click on Next
45. Fill the above as following:
    - Field Label: Status
    - Fill PickList Values: Pending, Confirmed, Delivered, Canceled
46. Click on Next → Next → Save.

Activity 15 : Creating Customer Name field in Vehicle_Customer object

47. Repeat the steps 1 & 2 from the activity 1
48. Select Data type as “Text”.
49. Click on Next
50. Fill the above as following:
    - Field Label: Customer_Name
51. Click on Next → Next → Save & New.

Activity 16 : Creating Email field in Vehicle_Customer object

52. Select Data type as “Email”.
53. Click on Next
54. Fill the above as following:
    - Field Label: Email
55. Click on Next → Next → Save & New

Activity 17 : Creating Phone field in Vehicle_Customer object

56. Select Data type as “Phone”.
57. Click on Next
58. Fill the above as following:

- Field Label: Phone

59. Click on Next → Next → Save & New

Activity 18 : Creating Address field in Vehicle_Customer object

60. Select Data type as “Text”.
61. Click on Next
62. Fill the above as following:
    - Field Label: Address
63. Click on Next → Next → Save & New

Activity 19 : Creating Preferred Vehicle Type field in Vehicle_Customer object

64. Select Data type as “Picklist”.
65. Click on Next
66. Fill the above as following:
    - Field Label: Preferred_Vehicle_Type
    - Fill PickList Values: Sedan, SUV, EV, etc.
67. Click on Next → Next → Save.

Activity 20 : Creating Vehicle Customer field in Vehicle_Test_drive object

68. Repeat the steps 1 & 2 from the activity 1
69. Select Data type as “Lookup Relationship”.
70. Click on Next
71. Fill the above as following:
    - Field Label: Vehicle _Customer
72. Click on Next → Next → Save & New.

Activity 21 : Creating Vehicle field in Vehicle_Test_drive object

73. Select Data type as “Lookup Relationship”.
74. Click on Next
75. Fill the above as following:
    - Field Label: Vehicle
76. Click on Next → Next → Save & New.

Activity 22 : Creating Test_Drive_Date field in Vehicle_Test_drive object

77. Select Data type as “Date”.
78. Click on Next
79. Fill the above as following:
    - Field Label: Test_Drive_Date
80. Click on Next → Next → Save & New.

Activity 23 : Creating Status field in Vehicle_Test_drive object

81. Select Data type as “Picklist”.
82. Click on Next
83. Fill the above as following:
    - Field Label: Status
    - Fill the Picklist value: Scheduled, Completed, Canceled
84. Click on Next → Next → Save.

Activity 24 : Creating Vehicle Customer field in Vehicle_Service_Request object

85. Repeat the steps 1 & 2 from the activity 1
86. Select Data type as “Lookup Relationship”.
87. Click on Next
88. Fill the above as following:
    - Field Label: Vehicle _Customer
89. Click on Next → Next → Save & New.

Activity 25 : Creating Vehicle field in Vehicle_Service_Request object

90. Select Data type as “Lookup Relationship”.
91. Click on Next
92. Fill the above as following:
    - Field Label: Vehicle
93. Click on Next → Next → Save & New.

Activity 26 : Creating Service Date field in Vehicle_Service_Request object

94. Select Data type as “Date”.
95. Click on Next
96. Fill the above as following:
    - Field Label: Service_Date
97. Click on Next → Next → Save & New.

Activity 27 : Creating Issue Description field in Vehicle_Service_Request object

98. Select Data type as “Text”.
99. Click on Next
100. Fill the above as following:
    - Field Label: Issue_Description
101. Click on Next → Next → Save & New.

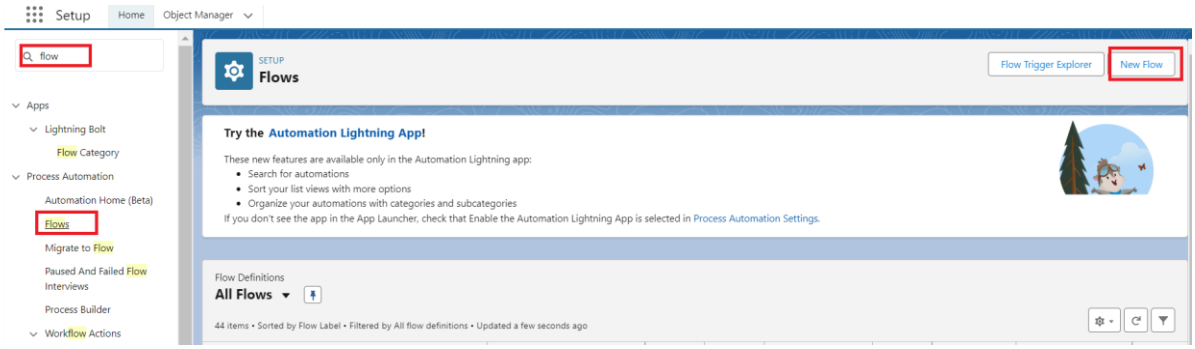
Activity 28 : Creating Status field in Vehicle_Service_Request object

102. Select Data type as “Picklist”.
103. Click on Next
104. Fill the above as following:
    - Field Label: Status
    - Fill Picklist value: Requested, In Progress, Completed
105. Click on Next → Next → Save & New.

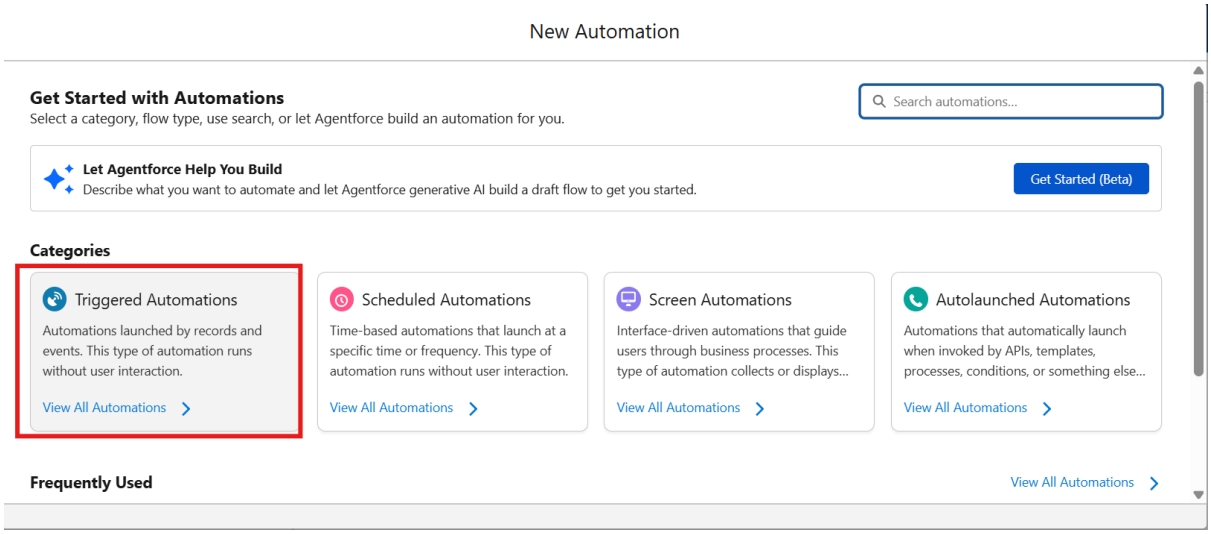
MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer’s location

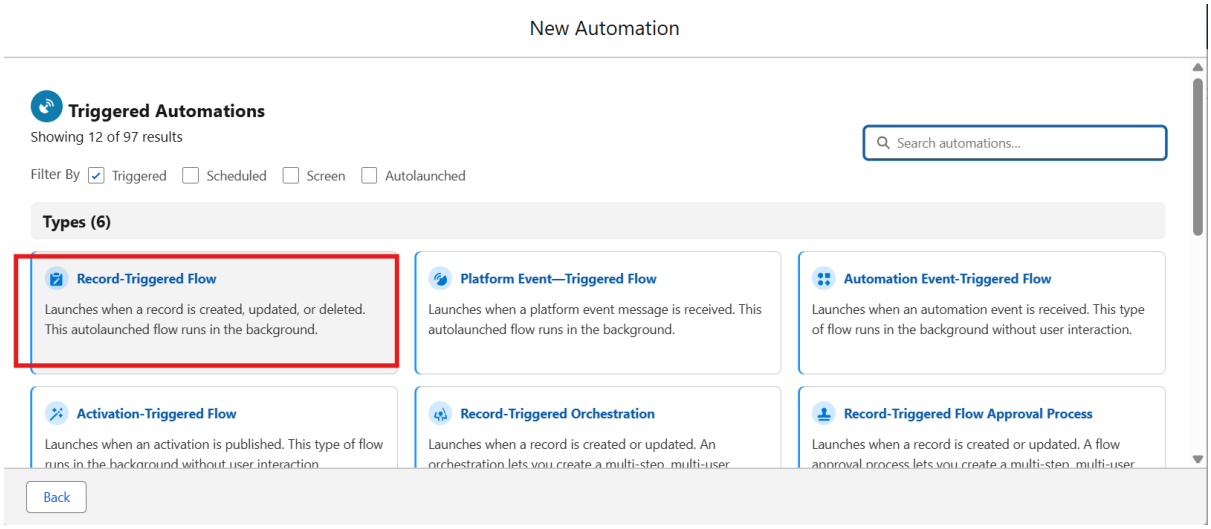
Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Triggered Automation From Flow.



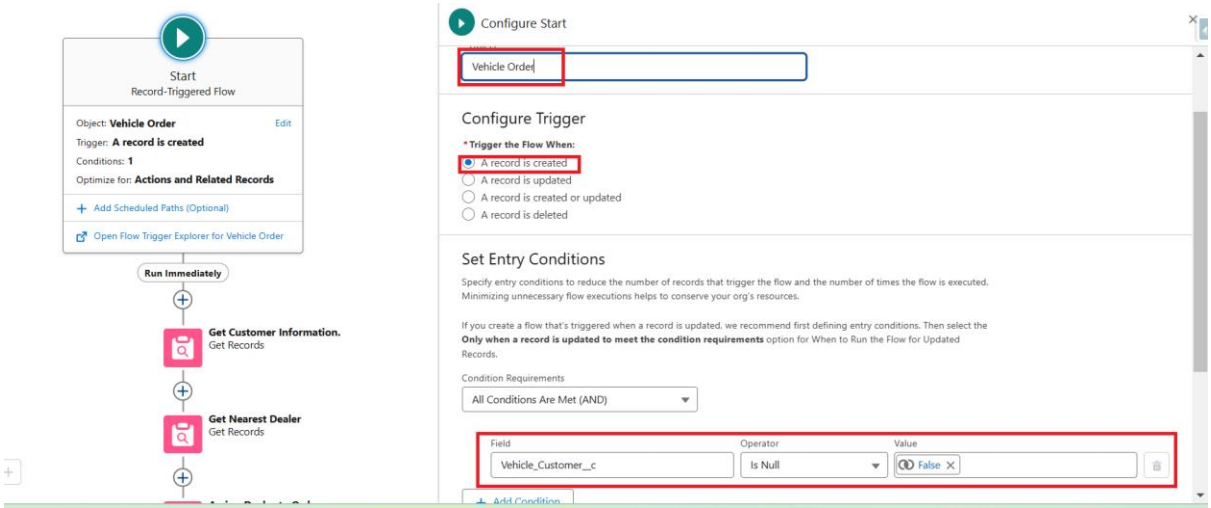
Step 3 : Select Record-Triggered Flow



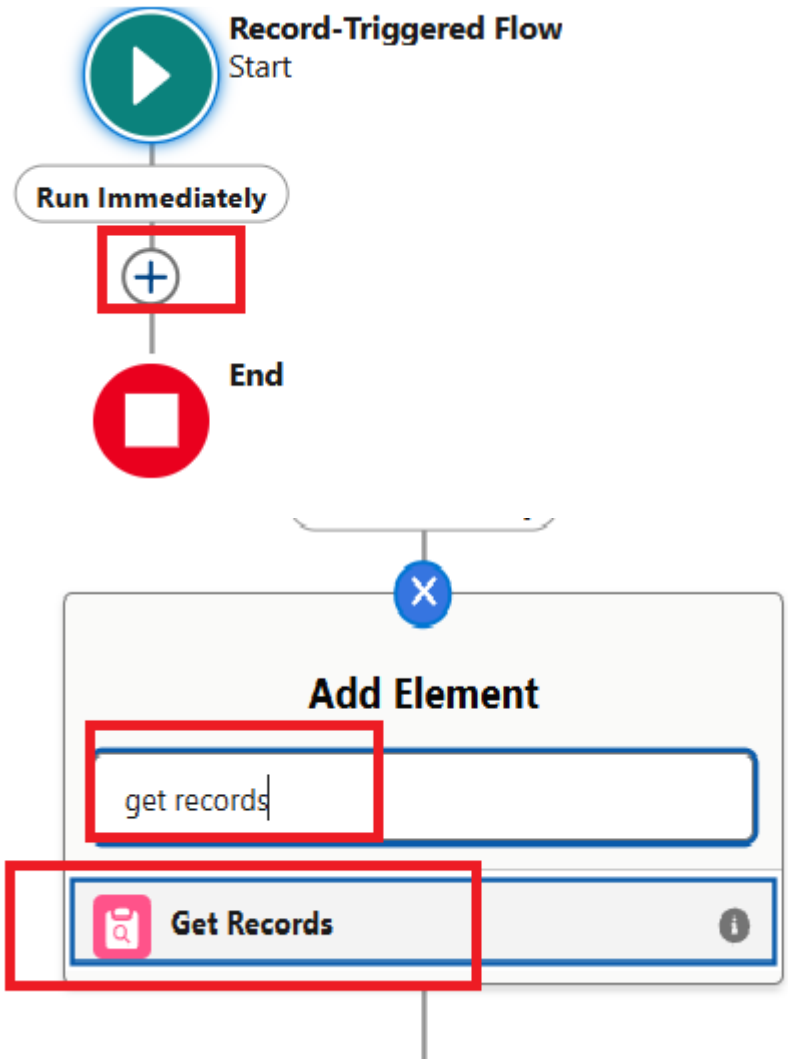
Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Pending

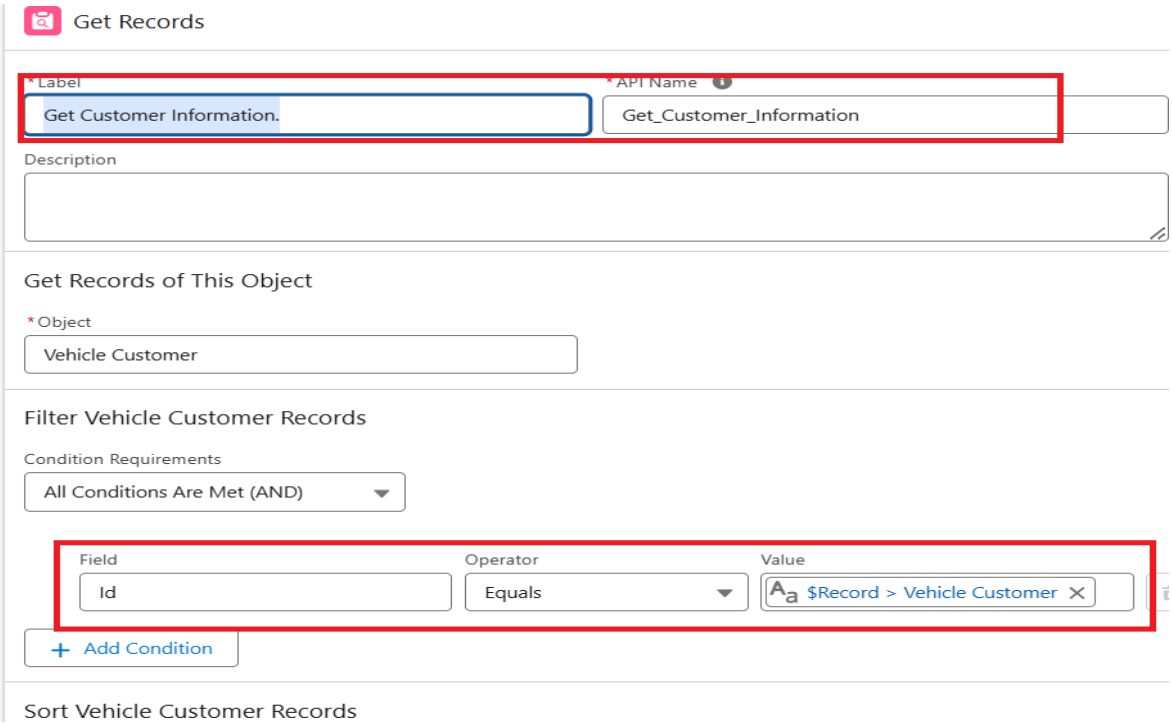


Step 5 : Click + → Select **Get Records**



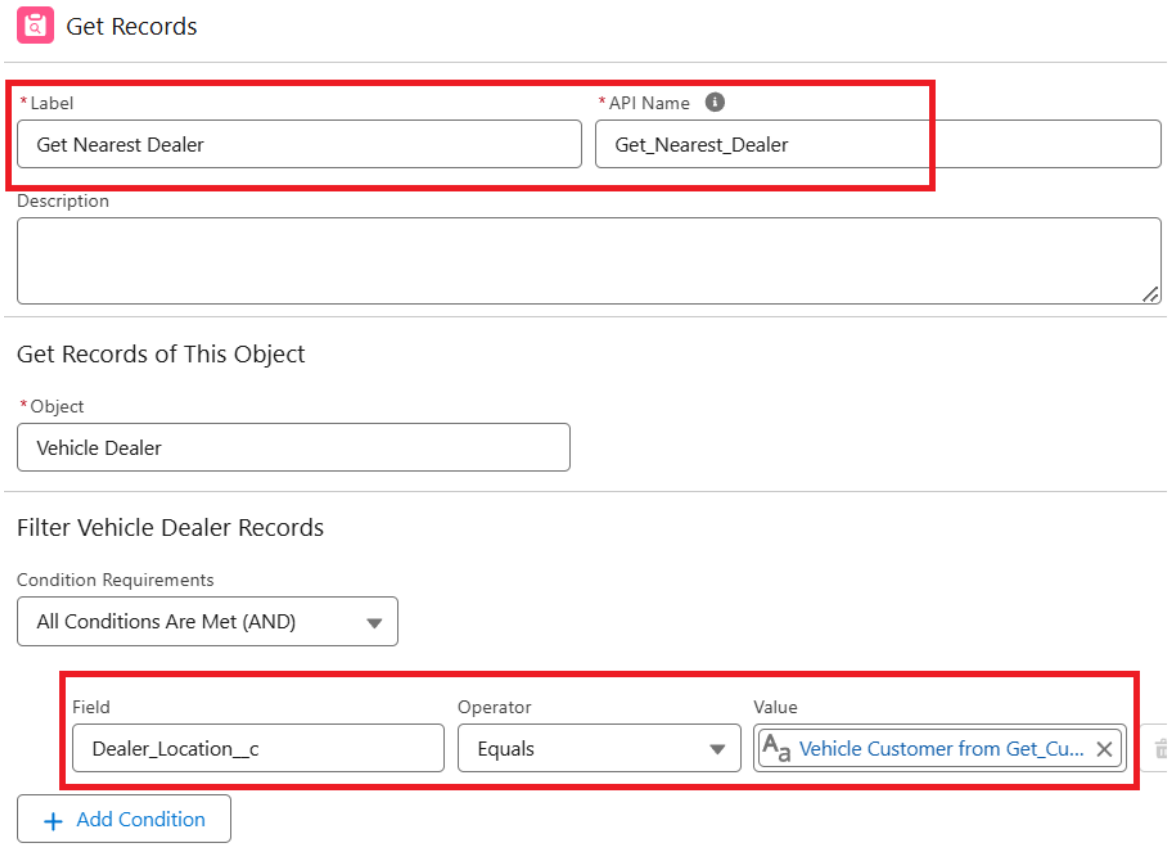
Step 6 : **Label :** Get Customer Information

Object : Vehicle Customer
Condition :
Field : Id
Operator : Equals
Value : {!$Record.Vehicle_Customer__c}



Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer
Object : Vehicle Dealer
Condition :
Field : Dealer_Location__c
Operator : Equals
Value : {!Get_Customer_Information.Address__c}

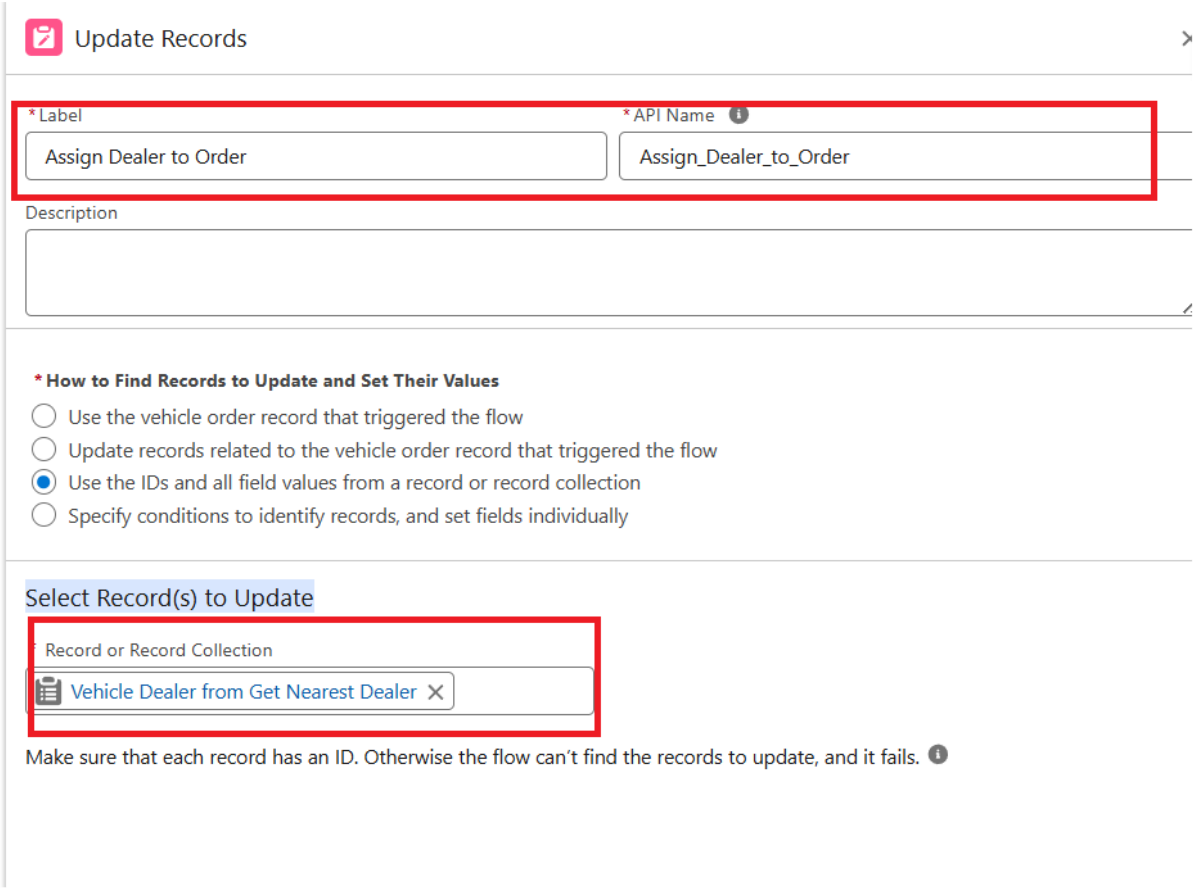


Step 8 : Click + → Select **Update Records**

Label : Assign Dealer to Order

*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}

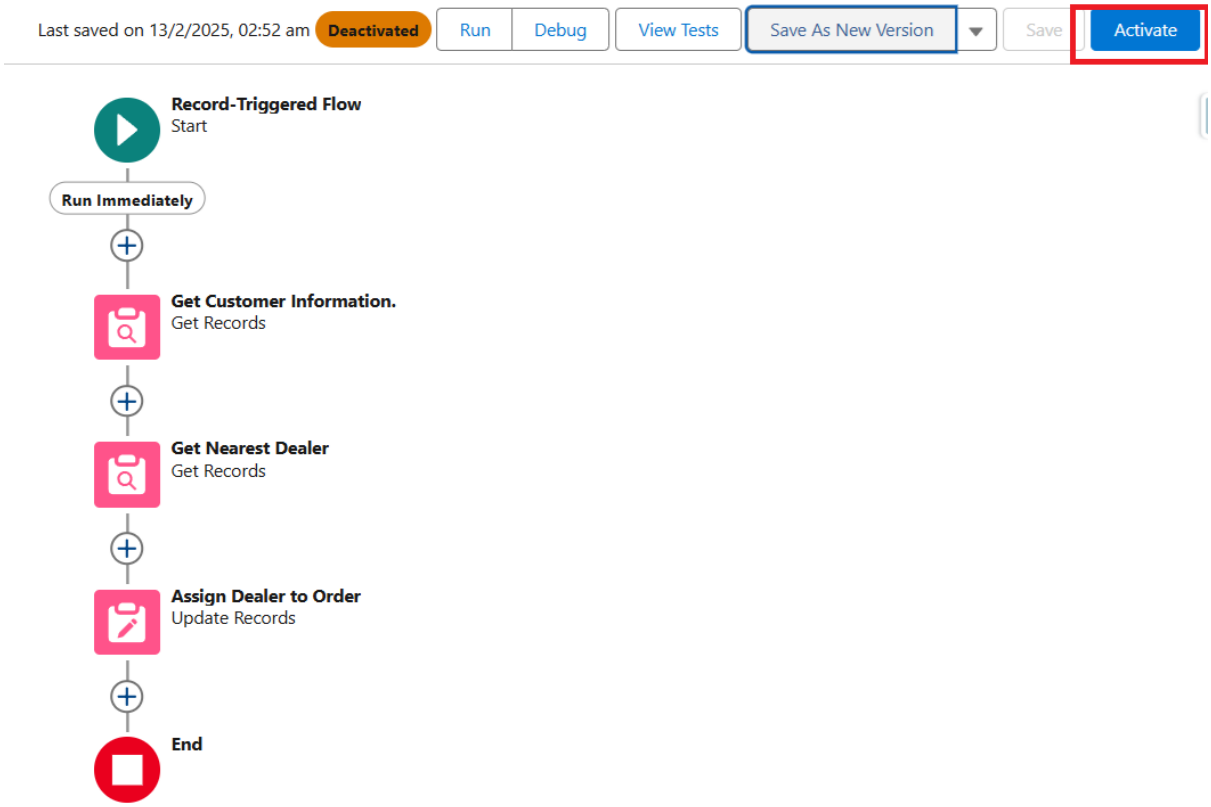


Step 9 : Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

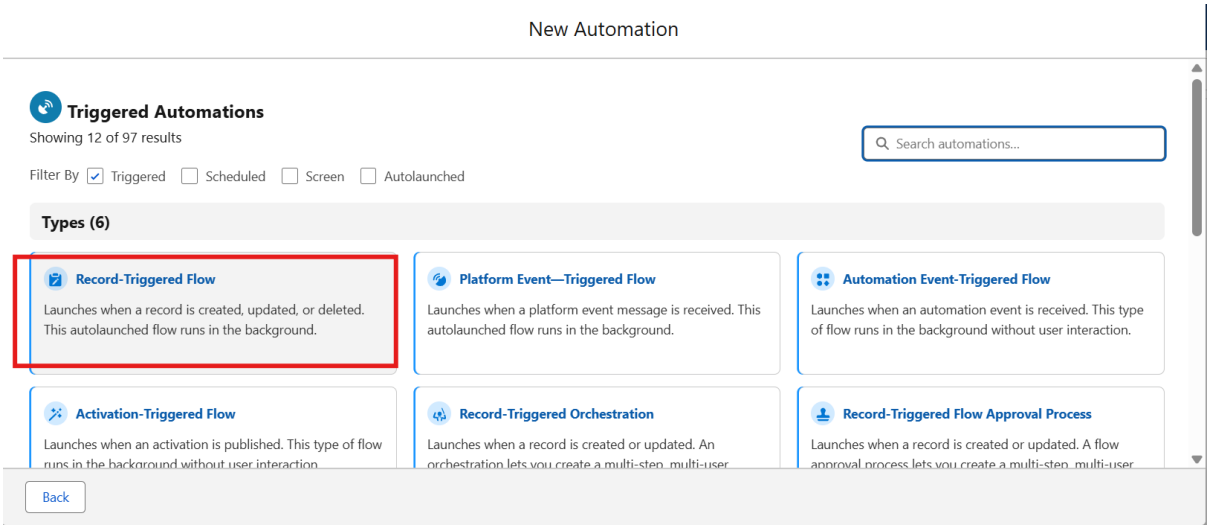


Step 10 : Activate Flow



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow



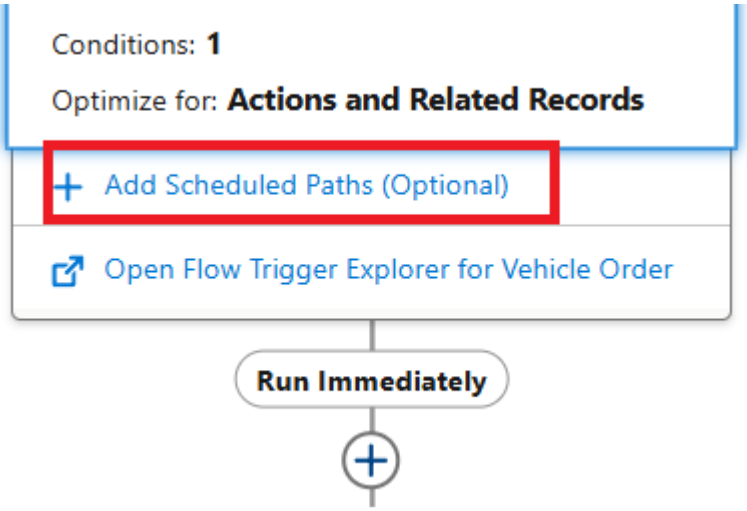
Step 2 : Select Vehicle Test Drive Object

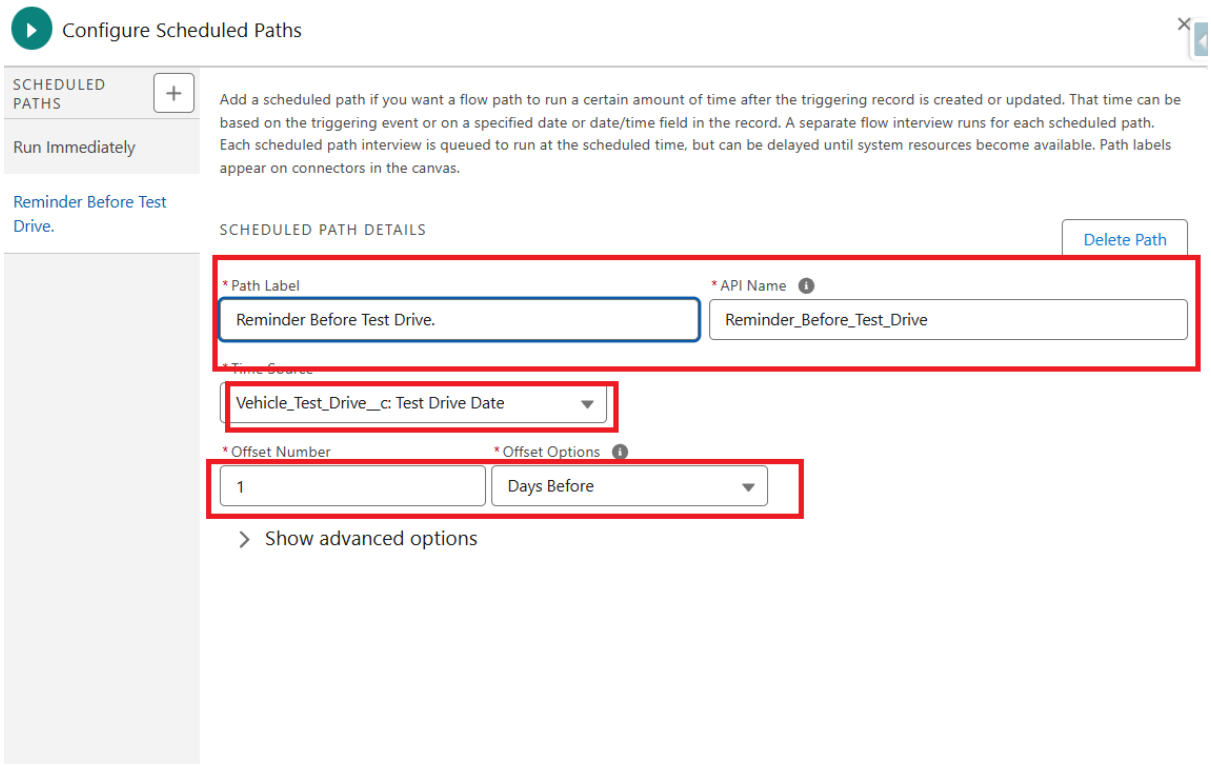
Trigger the Flow When : A record is created or updated

Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

Label: Reminder Before Test Drive.
Time Source: Test_Drive_Date__c
Offset Number: 1.
Offset Options: Days Before.
Click Done.

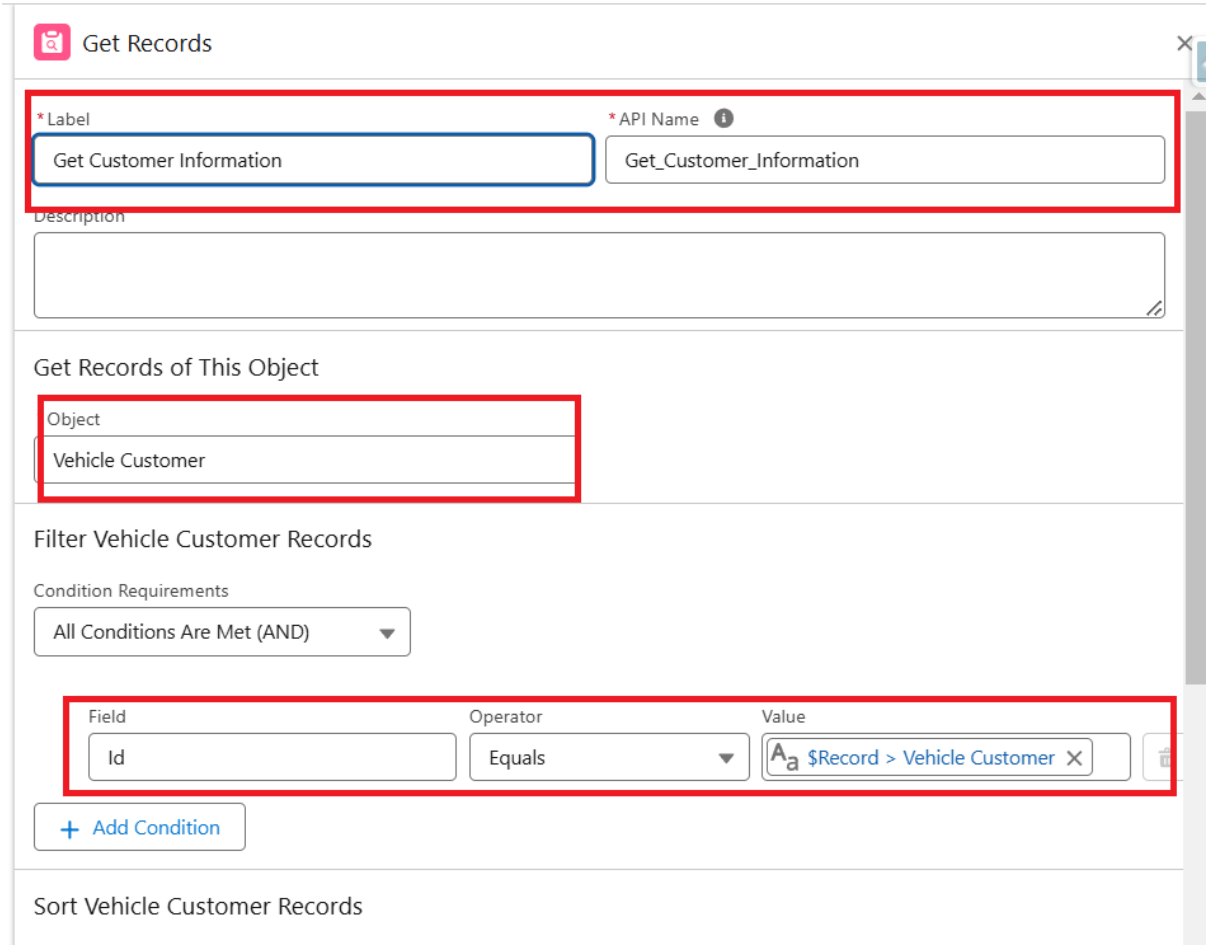




Step 4 :

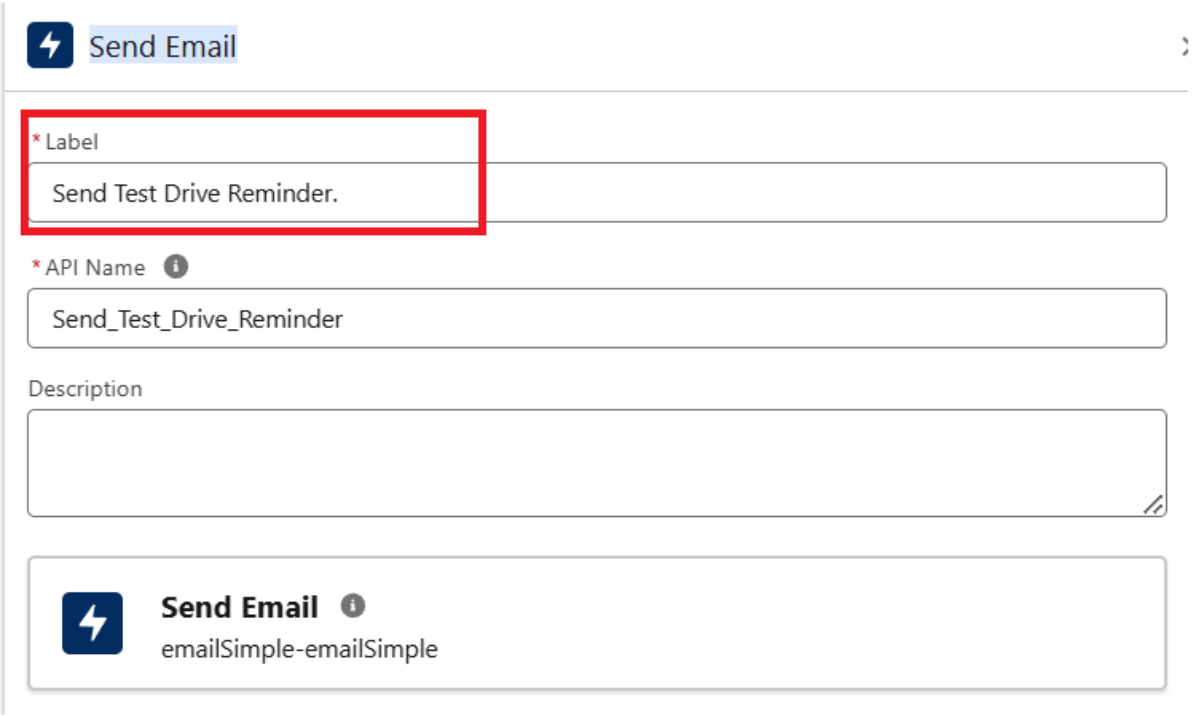
1. Click + Add Element → Get Records.
2. Label: Get Customer Information.

3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

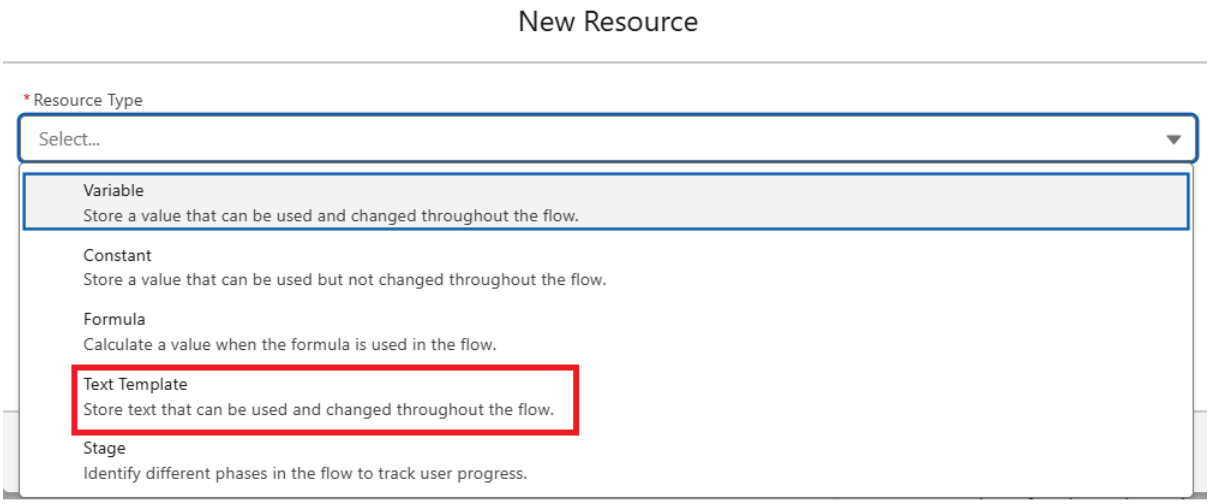


Step 5: Send Reminder Email

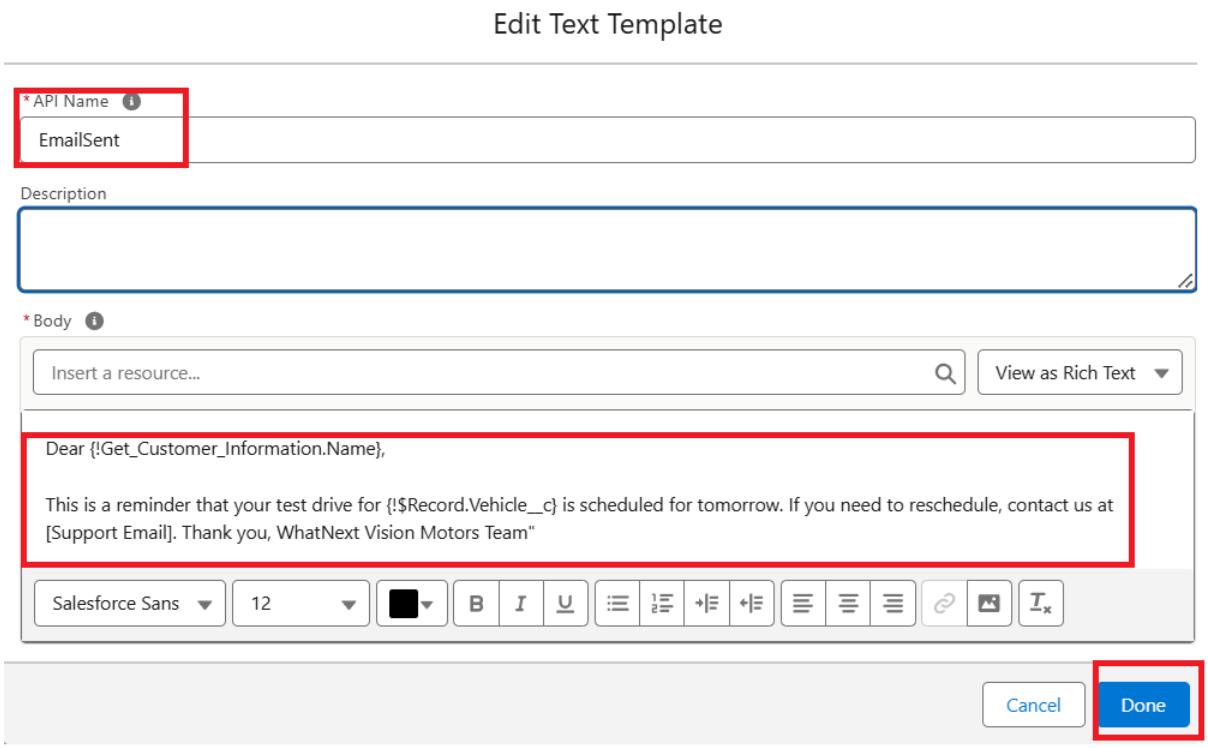
1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

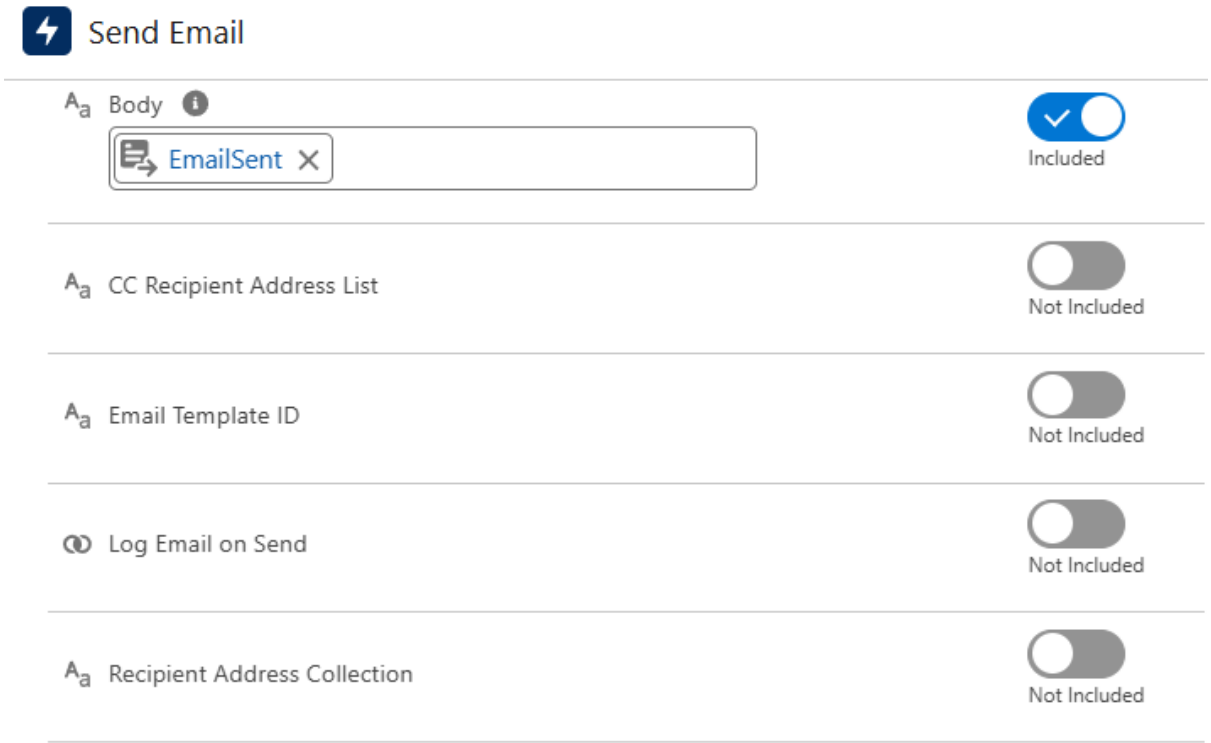


7. Body : Create Variable



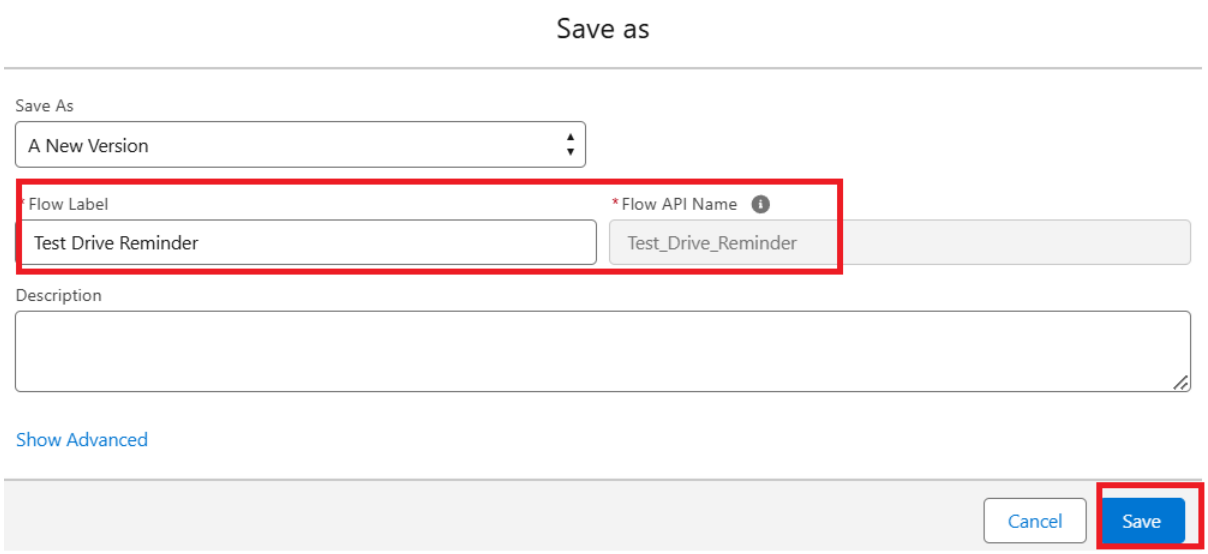
8 . Api Name : EmailSent



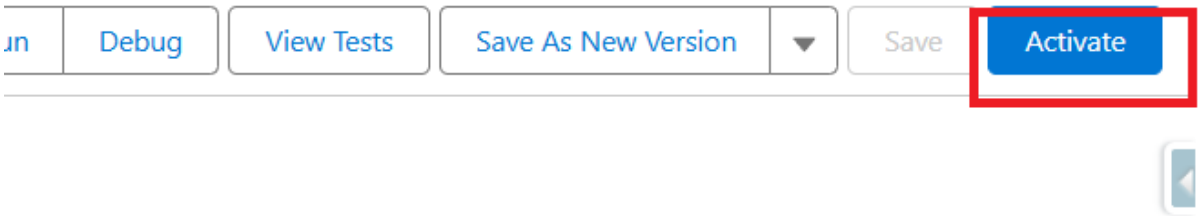


Step 6 : Click save

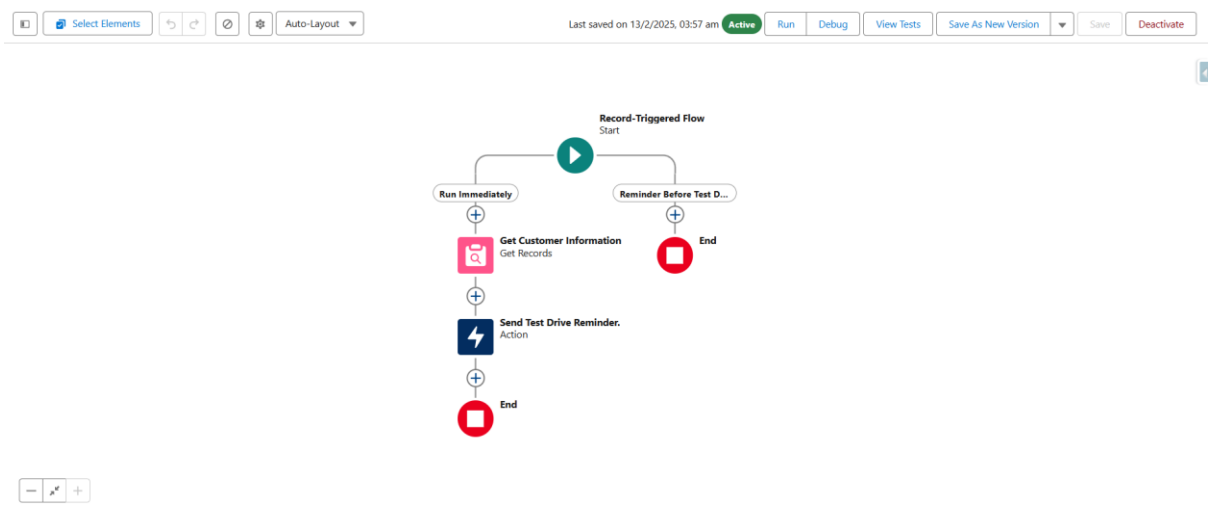
Label Name : Test Drive Reminder



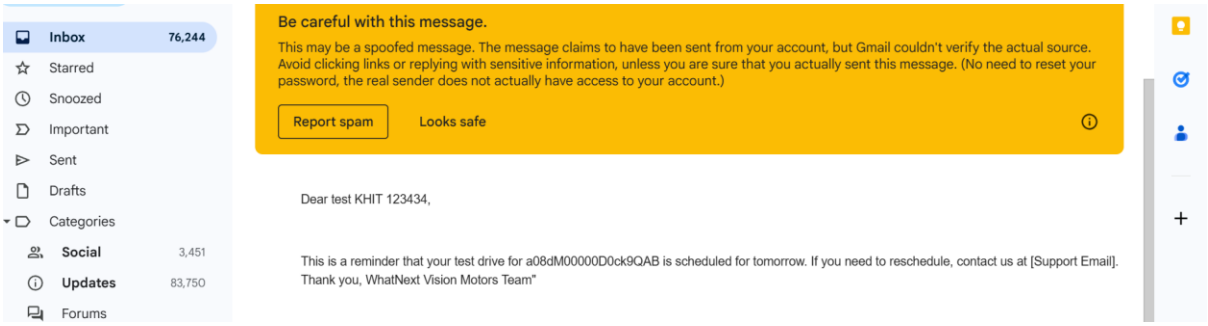
Step 7 : Activate Flow

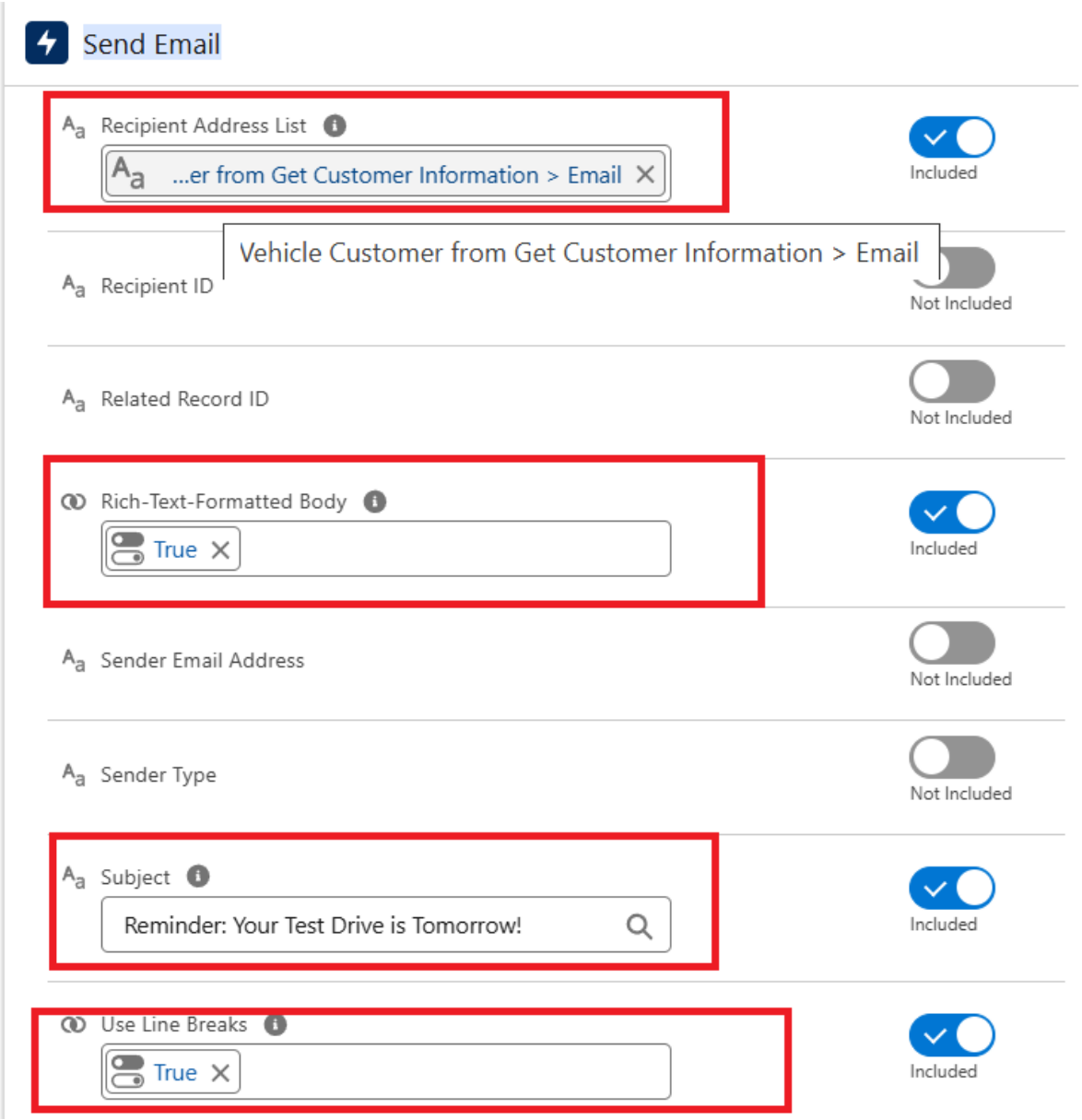


Flow Page After Creation :



Test your Flow :

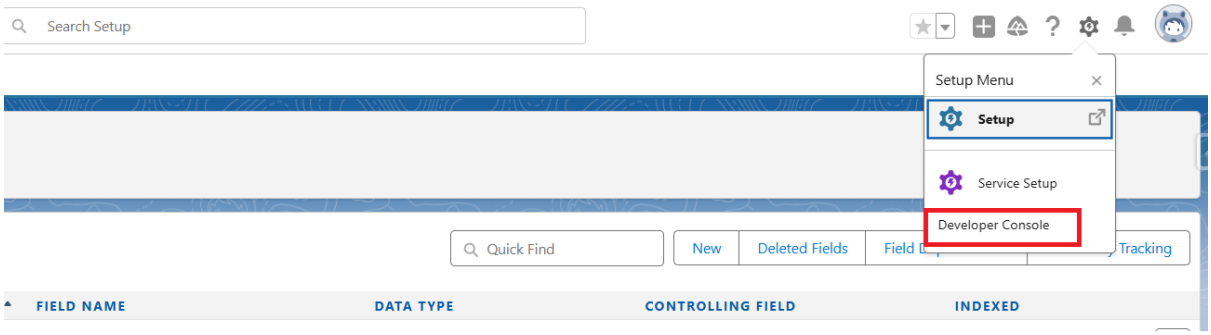




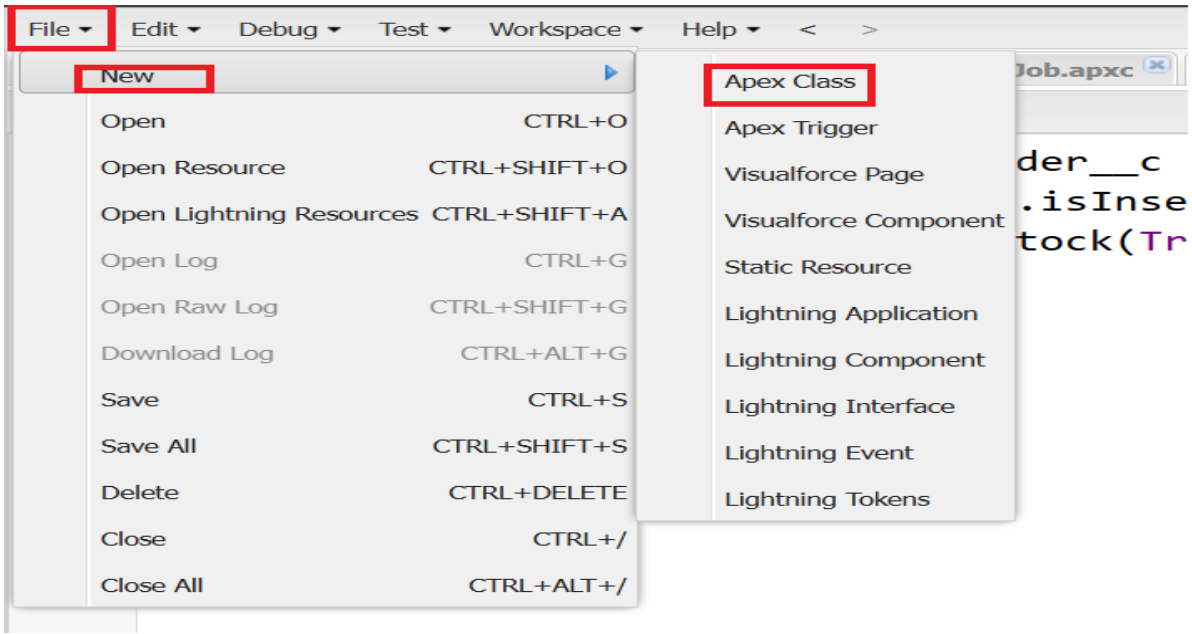
MileStone 7 :

Create Apex and Trigger Batch Jobs :

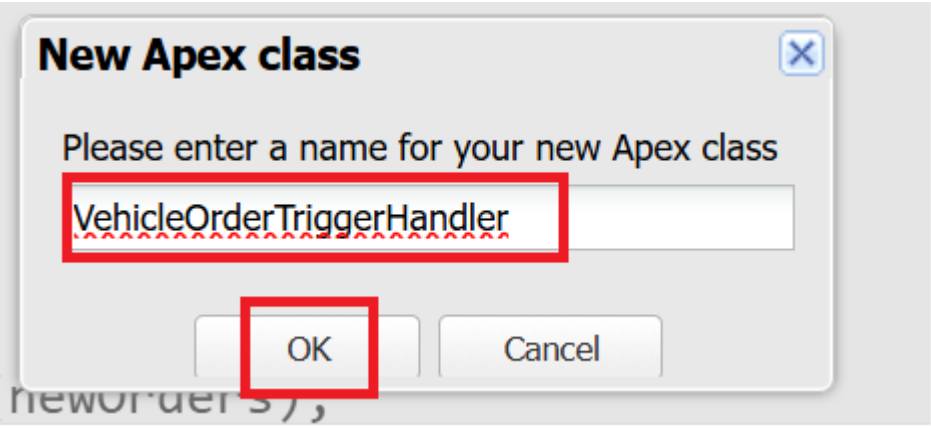
Step 1 : Click Developer Console From Gear icon



Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class



Step 4 : Write Apex Code

```apex
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
```

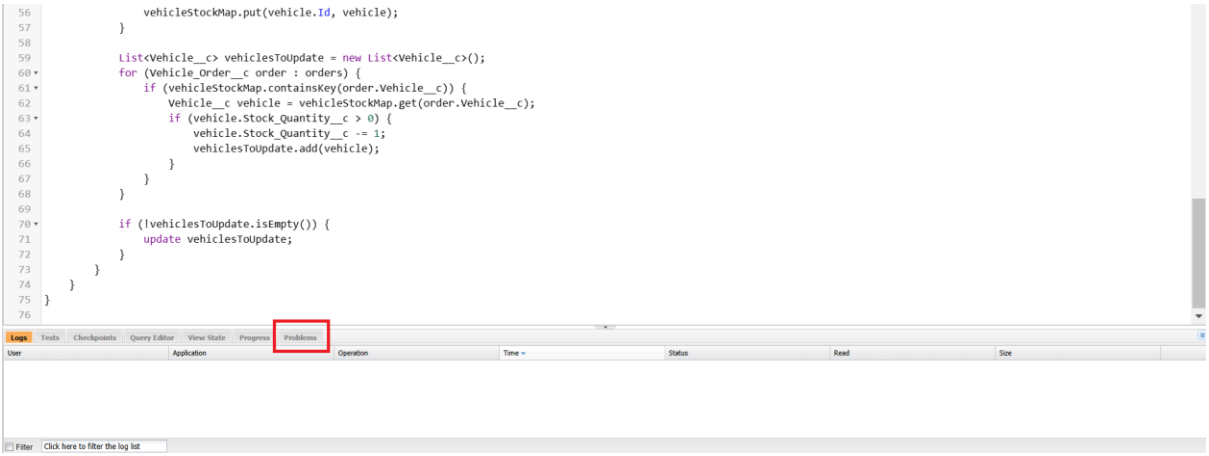
```apex
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
```



Source Code :

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
```

```
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be placed.');
                }
            }
        }
    }
}

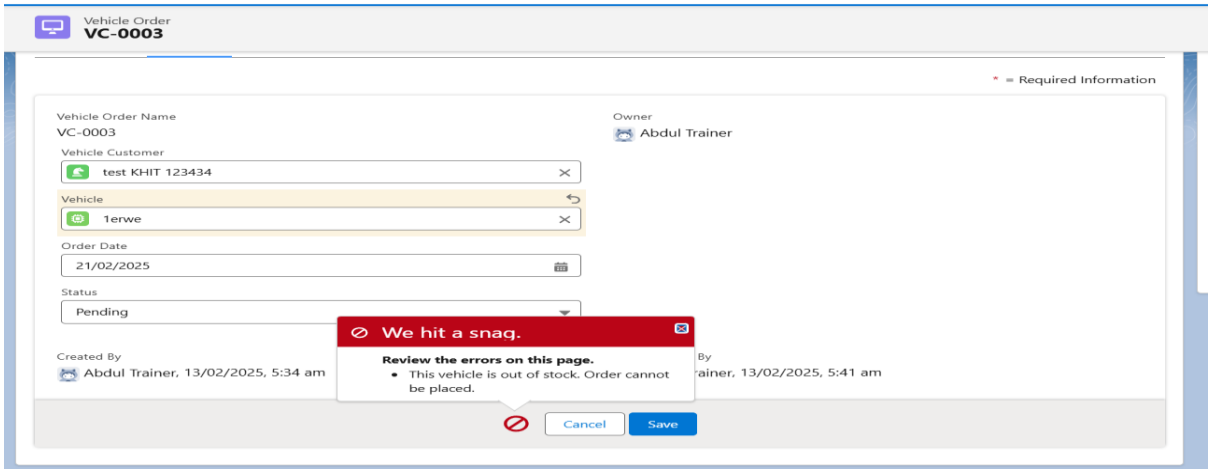
// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();

    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }

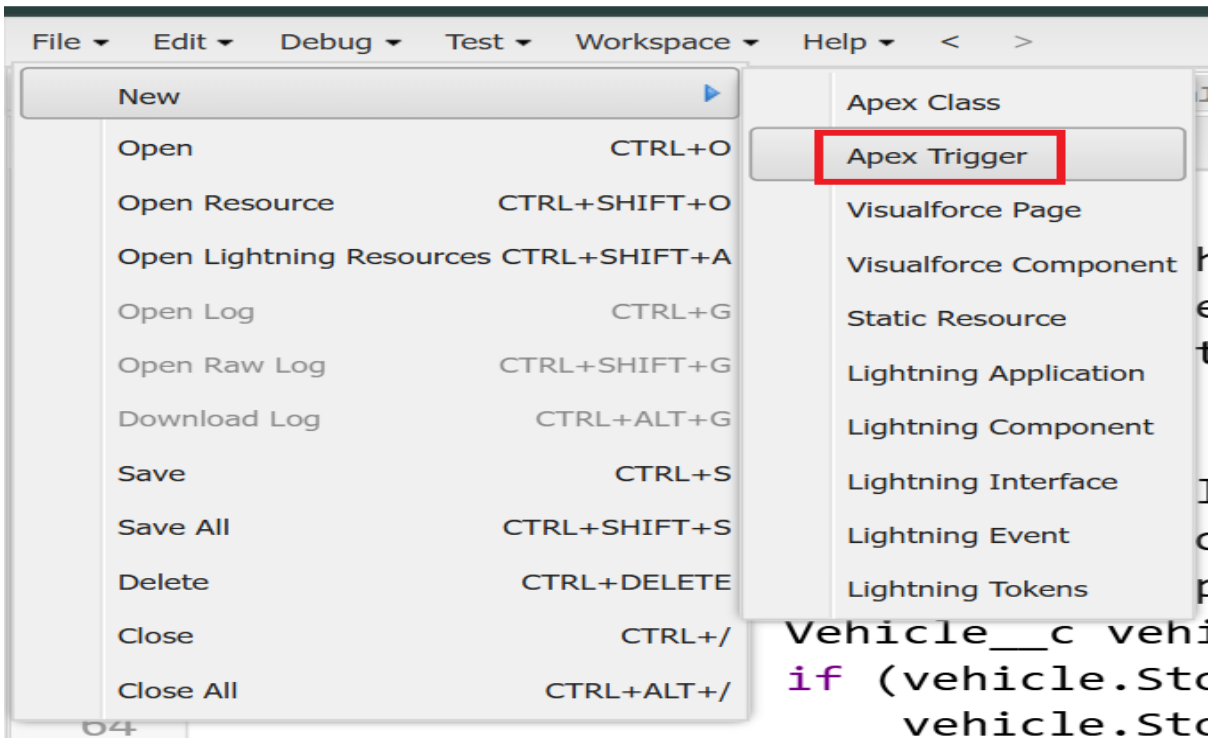
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }
```

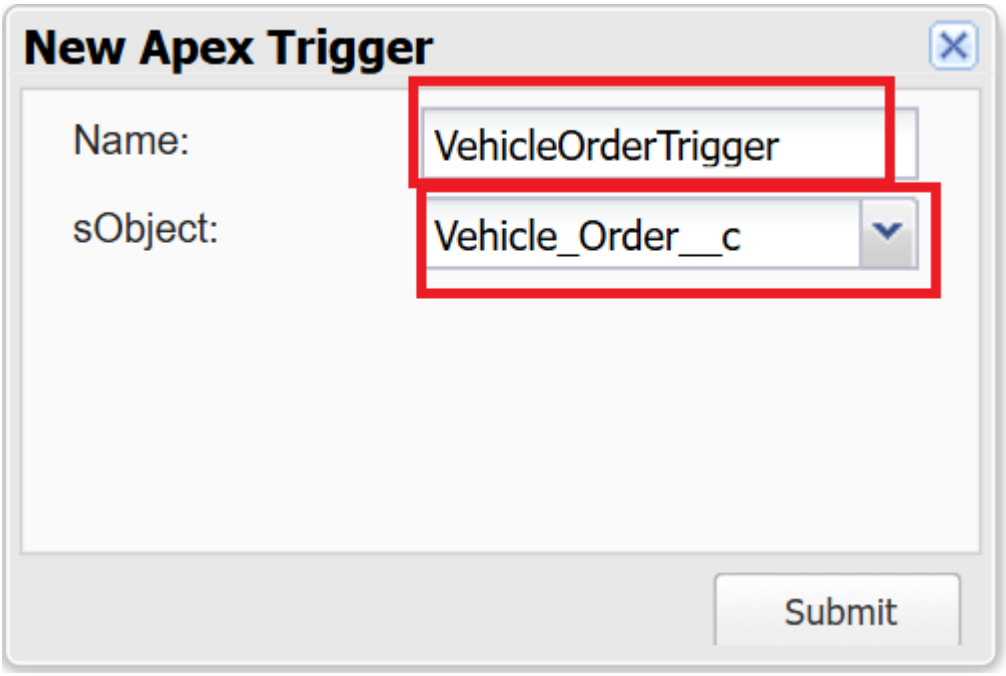
```
            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```



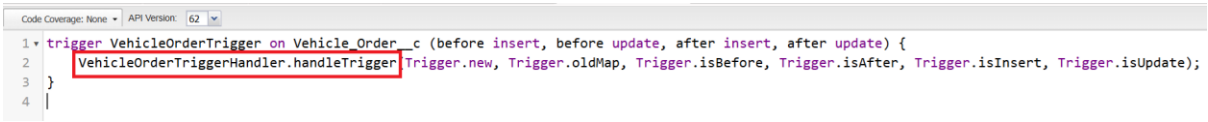
Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



Step 7 : Call Apex Class in Trigger Class

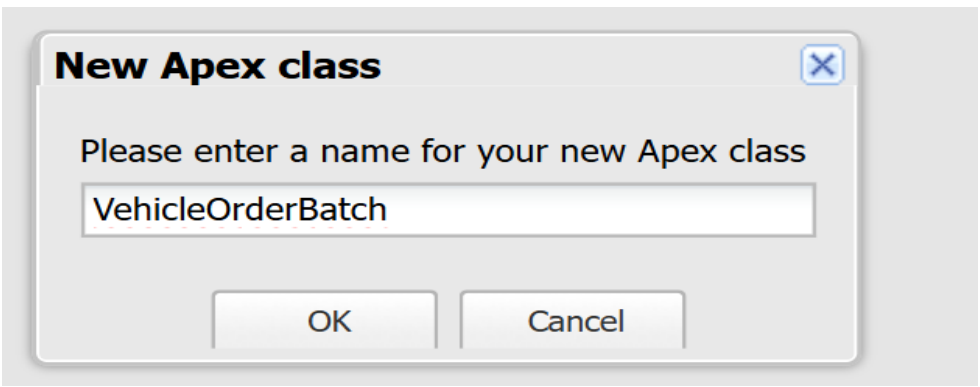


```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.



Code Coverage: None | API Version: 62

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

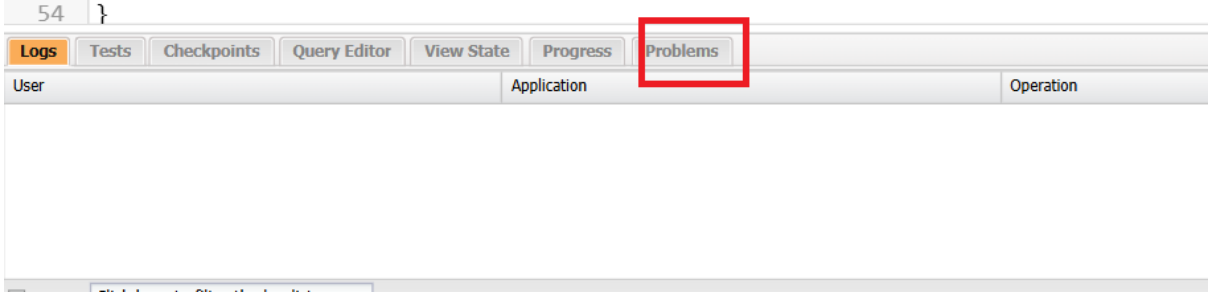
            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

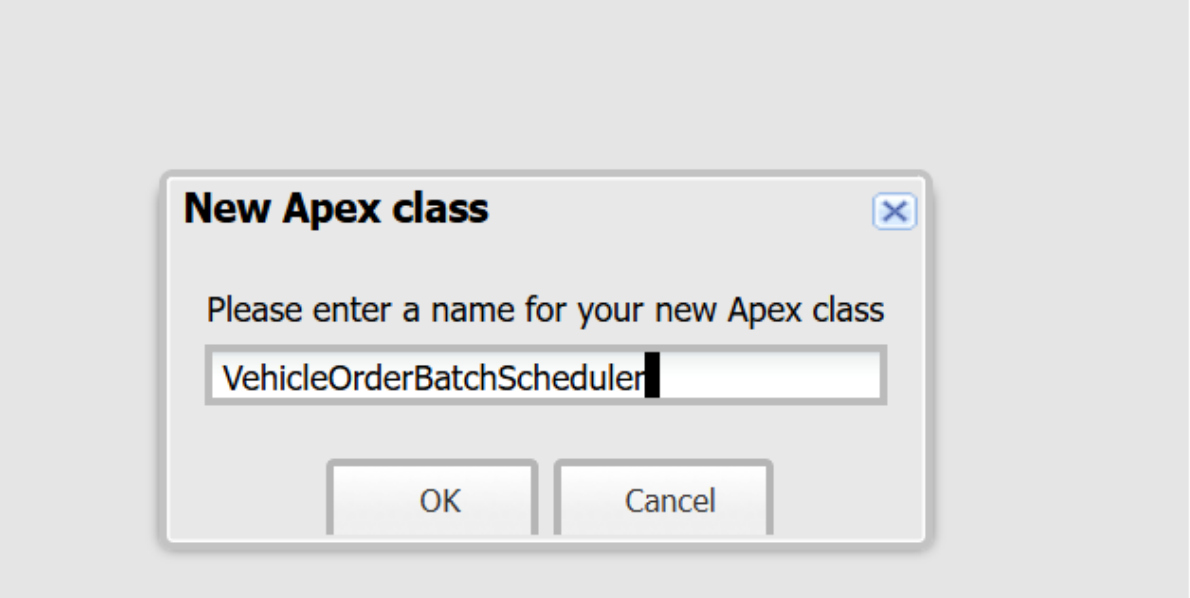
            if (!ordersToUpdate.isEmpty()) {
                update ordersToUpdate;
            }

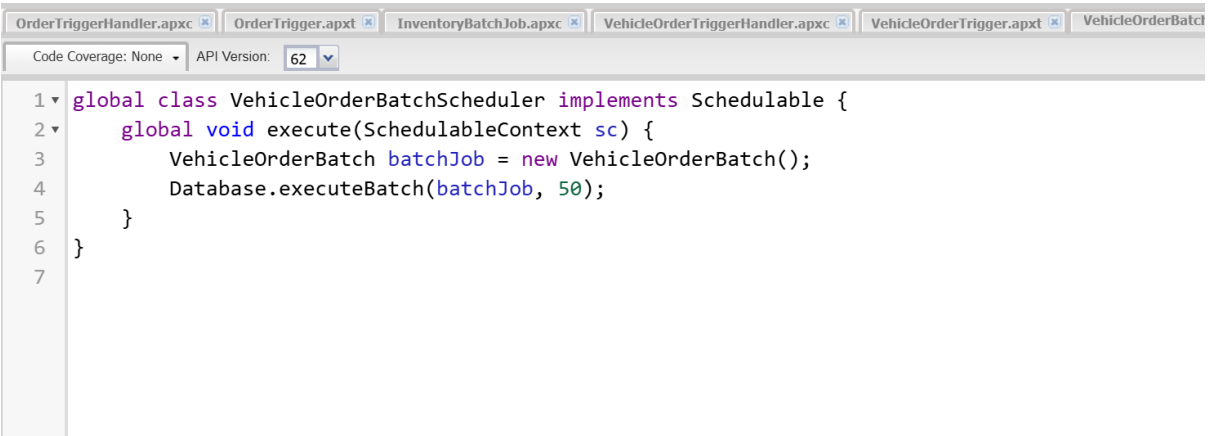
            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```



Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

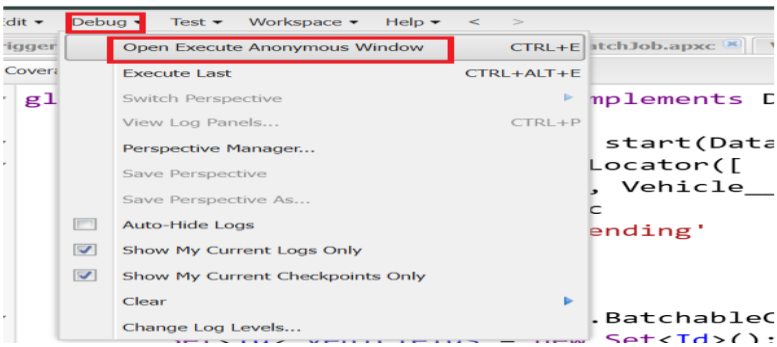


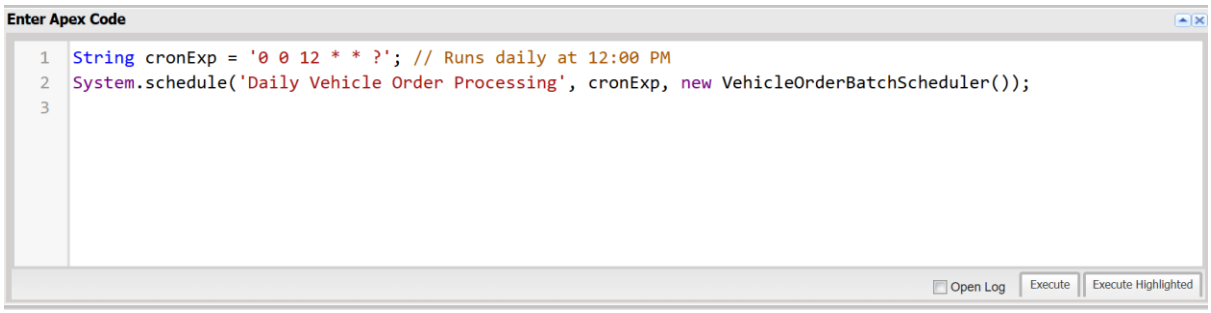


Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
   global void execute(SchedulableContext sc) {
      VehicleOrderBatch batchJob = new VehicleOrderBatch();
      Database.executeBatch(batchJob, 50); // 50 is the batch size
   }
}
```

Step 10 : **Schedule the Batch Job** To run the batch job **every night at midnight** :

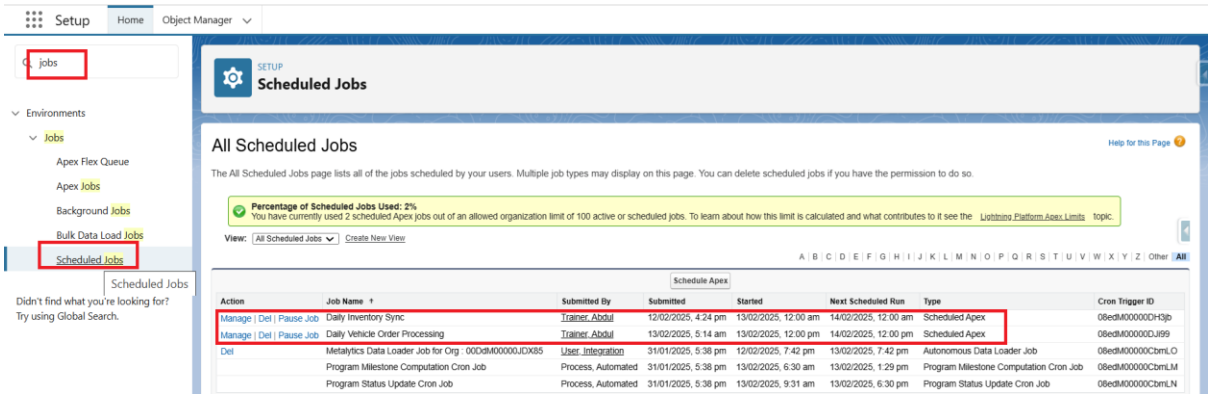




Source Code :
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running


Setup
Home
Object Manager
jobs
Environments
Jobs
Apex Flex Queue
Apex Jobs
Background Jobs
Bulk Data Load Jobs
Scheduled Jobs
Scheduled Jobs
Didn't find what you're looking for?
Try using Global Search.
SETUP
Scheduled Jobs
All Scheduled Jobs
Help for this Page
The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.
Percentage of Scheduled Jobs Used: 2%
You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the Lightning Platform Apex Limits topic.
View: All Scheduled Jobs
Create New View
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All
Schedule Apex
Action
Job Name
Submitted By
Submitted
Started
Next Scheduled Run
Type
Cron Trigger ID
Manage | Del | Pause Job
Daily Inventory Sync
Trainer, Abdul
12/02/2025, 4:24 pm
13/02/2025, 12:00 am
14/02/2025, 12:00 am
Scheduled Apex
08edM00000DH3jb
Manage | Del | Pause Job
Daily Vehicle Order Processing
Trainer, Abdul
13/02/2025, 5:14 am
13/02/2025, 12:00 pm
14/02/2025, 12:00 pm
Scheduled Apex
08edM00000DJI99
Del
Metalytics Data Loader Job for Org : 00DdM00000JDX85
User, Integration
31/01/2025, 5:38 pm
12/02/2025, 7:42 pm
13/02/2025, 7:42 pm
Autonomous Data Loader Job
08edM00000CbmLO
Program Milestone Computation Cron Job
Process, Automated
31/01/2025, 5:38 pm
13/02/2025, 6:30 am
13/02/2025, 1:29 pm
Program Milestone Computation Cron Job
08edM00000CbmLM
Program Status Update Cron Job
Process, Automated
31/01/2025, 5:38 pm
13/02/2025, 9:31 am
13/02/2025, 6:30 pm
Program Status Update Cron Job
08edM00000CbmLN

Thank you